

Carolus / RoboMaster S1 — Complete Technical Manual

August 27, 2026

Abstract

This manual is self-sufficient: following it from Chapter 3 onward requires no other document, inherited tutorial, or vendor PDF. It covers the complete path from unboxing a rooted RoboMaster S1 and a Raspberry Pi 4B to a running Carolus/uVGS-2 pose-estimation pipeline, then to driving the robot and running its beacon-docking behaviour (Chapter 11), using only the corrected, currently-working configuration of this project — not the historical or inherited versions of any script, launch file, or procedure. Every program listed here is the version that actually runs today.

Already set up? Chapter 2 is the five-minute routine for a normal session.

Contents

1	Introduction	2
1.1	What this manual covers	2
1.2	Hardware	2
1.3	Software stack	2
1.4	How the pieces fit together	3
1.5	What Carolus is	3
1.6	The four operating states	4
1.7	How to use this manual	4
2	Quick Start — A Typical Session	6
2.1	The five-minute version	6
2.2	Without the launcher, by hand	6
2.3	Is it working? Three checks from the lab PC	7
2.4	Shutting down	7
3	Powering On the Robot	8
3.1	Before you start	8
3.2	Power-on sequence	8
3.3	Confirming the robot is powered on	9
3.4	Confirming the robot is rooted	9
3.5	Which camera is mounted	9
4	Rooting the S1	10
4.1	What the hack does	10
4.2	Step-by-step procedure	10
4.2.1	What you need	10
4.2.2	Procedure	11
4.3	A pitfall to know before rooting	12
5	Setting Up the Raspberry Pi	13
5.1	Powering the Pi — two USB-C ports, only one of them right	13
5.2	Flashing Ubuntu Server	13
5.3	Desktop environment (optional)	14
5.4	Enabling headless SSH access	14
5.5	Building Python 3.7 from source	15

5.6	Virtual environment and the RoboMaster SDK	15
5.7	Installing ROS Noetic	16
5.8	Getting this project’s code onto the Pi	17
5.9	A pitfall to expect at first launch	17
6	Network Setup (RNDIS)	18
6.1	Bringing up the link	18
6.1.1	If you intend to automate this over SSH	18
6.2	Verifying the link	19
6.3	A quick automated alternative	19
6.4	Lab PC hostname resolution	19
6.5	Summary of addresses used in this manual	19
7	The Camera and Robot Bridge	20
7.1	Design: a single SDK connection	20
7.2	Starting the bridge	20
7.3	What it connects to	20
7.4	Verifying it is running	22
7.5	Gimbal yaw: two signals, two different meanings	22
7.6	The internal camera vs. an external D455	22
8	Camera Calibration	23
8.1	Current values in use	23
8.2	Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox	23
8.3	Known limitation: no confirmed auto-exposure control	24
8.4	Alternative / complementary method: Kalibr	24
8.4.1	Target	25
8.4.2	Installing Kalibr	25
8.4.3	Capturing data and running the calibration	25
8.4.4	Reading the result	26
8.5	Camera-IMU time offset	26
9	Building and Launching Carolus	27
9.1	Where Carolus runs	27
9.1.1	The two machines, side by side	28
9.1.2	Measured, 2026-08-04 — and what the numbers do <i>not</i> say	28
9.1.3	A note on visual-inertial navigation	29
9.1.4	MINS on the ground station (lab PC)	29
9.2	Getting this project’s code onto the machine that will build it	31
9.2.1	Setting up an SSH key	31
9.3	Installing the workspace’s remaining dependencies	32
9.4	What the package builds: two targets, not one	33
9.5	Building on Ubuntu 20.04 (the Raspberry Pi — normal case)	33
9.6	Building on Ubuntu 22.04 (lab-PC fallback only)	34

9.6.1	Fix 1: glibc symlinks	34
9.6.2	Fix 2: libpython3.8 symlink	34
9.6.3	Fix 3: Boost 1.71 (real libraries, not symlinks)	35
9.6.4	Fix 4: ICU 67	35
9.6.5	Building, after the four fixes	35
9.6.6	Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)	36
9.6.7	If an additional ROS package needs to be built from source	36
9.7	Launching Carolus	36
9.7.1	The hardware profile	36
9.7.2	Launching	37
9.7.3	The pose timestamp: <code>stamp_from_acquisition</code>	38
9.8	Verifying it is running	38
9.9	Beacon geometry and blob-filtering parameters	39
9.9.1	The pose filter	40
9.9.2	Operating rules for the beacon	42
10	Converting a Carolus Pose into the ROS Frame	44
10.1	Why this is harder than it looks	44
10.2	Step 1 — what Carolus actually publishes	44
10.3	Step 2 — what TF2 expects	45
10.4	Step 3 — the change of basis, and why it is proper	45
10.4.1	Carolus’s camera frame, stated definitively	45
10.4.2	Carolus is NOT the Intel D455 convention, and NOT the SVGS one	45
10.4.3	The change-of-basis matrix	46
10.5	Step 4 — a rotation is re-based by conjugation, not permutation	46
10.6	The result	47
10.6.1	The numeric value of b , and how it is obtained	47
10.6.2	Converting the result to roll, pitch and yaw	48
10.7	Cross-checking the result against an independent implementation	49
10.8	Validation status	49
11	Operating the Robot: the Launcher	51
11.1	The launch tabs, in order	51
11.2	Driving the robot by hand	52
11.3	The main controls	53
11.4	Session logs on disk	54
11.5	Shutting down, and why it matters	54
11.6	Motion commands from Carolus readings: beacon docking (removed from the launcher)	54
12	MINS on the Ground Station	55
12.1	What this is, and what it is not	55
12.2	Building on the lab PC (Ubuntu 22.04)	55
12.3	Running the bundled simulation (proof of life)	57
12.4	Config and launch file reference	57

12.5	Recording a rosbag for a real replay	58
12.6	Plotting the estimate against ground truth	58
12.7	sqrTVINS — the comparison partner	59
13	Carolus with a USB Webcam	60
13.1	Why this exists	60
13.2	Installing the camera driver	60
13.3	Running it	61
13.4	The Carolus config	61
13.5	Reading the result	61
14	Troubleshooting	63
14.1	Nothing responds: the robot ignores every command	63
14.2	No camera preview, and the camera/gimbal buttons do nothing	63
14.3	Choosing the robot mode: the camera wanders off target	64
14.4	The robot slowly rotates on its own	65
14.5	The chassis ignores every command while everything else works	66
14.6	The Pi will not turn on	67
14.7	rostopic from the lab PC finds nothing	67
14.8	The camera publishes but /pose never does	67
14.9	The keyboard does not drive the robot	68
14.10	/imu is silent	68
14.11	The build fails	68
14.12A	A plain python3 script dies on import cv2	68
14.13	Messages you can safely ignore	68

Chapter 1

Introduction

1.1 What this manual covers

This is the complete, self-contained setup and operation manual for the Carolus/RoboMaster S1 platform: a DJI RoboMaster S1 ground robot, rooted to expose the RoboMaster EP SDK, carrying a Raspberry Pi 4B at the turret position, running Carolus (also called uVGS-2) — a 4-LED beacon pose-estimation system — to autonomously search for, align to, approach, and dock at a fixed beacon.

Everything in this manual has been executed and verified on this exact hardware. Where a step has not been physically re-verified on this specific unit but is inherited from a documented, working precedent (e.g. a rooting procedure that predates this project), that is stated explicitly at the point it occurs.

1.2 Hardware

- **Robot:** DJI RoboMaster S1, rooted via the `s1_sdk_hack_v0.0.5` community hack, which exposes the RoboMaster EP SDK (chassis drive, gimbal control, onboard camera, wheel odometry, IMU) over a local API not officially released for the S1.
- **Companion computer:** Raspberry Pi 4B, mounted at the turret, connected to the robot over USB via RNDIS. The Pi holds the single connection to the robot’s SDK, runs the camera/beacon-tracking node, and **runs Carolus itself**, so the full perception pipeline is onboard and the robot needs no network link to estimate its pose (Section 9.1).
- **Lab PC:** runs the operator GUI and builds/edits the same Carolus code for development, but does not operate the robot with it — not physically connected to the robot, communicating with the Pi over the local network.
- **Camera:** the S1’s own internal camera (built into the gimbal/head), streamed over the SDK. This project does not use an external Intel RealSense D455 — some inherited reference material describes a D455-based pipeline (see Chapter 8’s calibration discussion); that material is noted where it applies to calibration *technique*, not to this robot’s actual camera hardware.

1.3 Software stack

- **Operating systems:** Raspberry Pi runs Ubuntu 20.04.6 LTS (aarch64). The lab PC runs Ubuntu 22.04.5 LTS (x86_64) — not the officially supported target for ROS Noetic, which requires local

workarounds documented in Chapter 9.

- **Middleware:** ROS Noetic (1.17.4) on both machines.
- **Robot control:** the official `robomaster` Python SDK, requiring Python 3.7 specifically.
- **Vision:** `Carolus/uVGS-2`, a C++ ROS node built on OpenCV, Eigen3, and Ceres Solver, performing 4-LED blob detection and a Perspective-4-Point (P4P) pose solve.

1.4 How the pieces fit together

Three machines, one data path. The robot never talks to the lab PC directly — everything goes through the Pi, which holds the only SDK connection.

RoboMaster S1		Raspberry Pi 4B		Lab PC
camera, chassis, gimbal	USB / RNDIS	SDK bridge, Carolus, ROS master	local net	launcher (GUI), development
[]	<----->	[]	<----->	[]
192.168.42.2		192.168.42.3 / 192.168.0.103		192.168.0.100

Inside the Pi, the pipeline is a straight line from pixels to a pose:

camera frames	-->	blob detection	-->	P4P solve (Ceres)	-->	/pose 6-DOF	-->	control SEARCH / ALIGN / APPROACH
		4 LEDs						
		+						+
				/carolus/cmd_vel		->	back to the robot	

`rm_cam_beacon.py` (Chapter 7) owns both ends — it publishes the frames and it is the only thing that sends motion commands to the robot. `Carolus` (Chapter 9) owns the middle.

1.5 What Carolus is

`Carolus` (internally named `uVGS-2`, Unified Visual Guidance System version 2) is a monocular pose-estimation system originally developed for NASA’s *Astrobee* free-flying robot aboard the International Space Station, derived from the earlier `SVGS` system. It detects four illuminated beacon points (LEDs) in a camera image, identifies which detected blob corresponds to which known 3D point on the physical beacon, and solves a Perspective-4-Point optimization (via Ceres Solver, not OpenCV’s `solvePnP` family) to recover the beacon’s full 6-DOF pose relative to the camera.

Carolus’s camera frame follows the standard computer-vision convention — X right, Y down, Z forward — which is *right-handed*, as is ROS’s REP-103 body frame (X forward, Y left, Z up). Converting between them is therefore an ordinary change of basis between two right-handed frames, and is necessarily a *proper* rotation (determinant +1). Chapter 10 derives the conversion in full and explains why the operation is a similarity transform rather than a component-wise permutation of the quaternion’s vector part.

Correction, 2026-08-11. Earlier revisions of this manual stated that Carolus’s native frame is *left-handed*, and that the conversion to REP-103 is therefore improper (determinant -1). **That was wrong.** REP-103 defines both the body frame and the camera “_optical” frame it names, and states that all its frames are right-handed; a change of basis between two right-handed orthonormal frames is orientation-preserving by the definition of a rotation, so determinant $+1$ is the only possibility. The claim is corrected here rather than quietly deleted, because a reader who saw the earlier version may have carried the improper-conversion idea forward into their own work.

1.6 The four operating states

Once running, the robot’s autonomous behavior is a state machine:

SEARCH The gimbal sweeps without moving the chassis, looking for the beacon. If not found from the current heading, the chassis rotates to a new heading and the gimbal sweeps again (a “fan search” pattern).

ALIGN Once the beacon is detected, the chassis rotates in place until it faces the beacon.

APPROACH The gimbal tracks the beacon while the chassis advances along the same heading.

STOP Triggered once within a target stopping distance of the beacon.

A separate, more deliberate **docking** behavior performs a calibrated, verified approach-and-stop sequence rather than the lighter continuous SEARCH/ALIGN/APPROACH loop (Section 11.6).

1.7 How to use this manual

If the hardware is already set up, go straight to Chapter 2 — it is the five-minute routine for a normal session, and it is what you will use every day after the first one.

From scratch, follow the chapters in order: power the robot on and confirm it is rooted (Chapter 3), root it if it is not (Chapter 4), set up the Raspberry Pi (Chapter 5), bring up the network link (Chapter 6), bring up the camera pipeline (Chapter 7), calibrate the camera (Chapter 8), build and launch Carolus (Chapter 9), then drive the robot (Chapter 11).

Do not calibrate the camera on a first run. The values already in the repository work on this robot. Chapter 8 is for a different camera, or for when poses look physically wrong — skipping it initially is the right call, not a shortcut.

What you need, and how long it takes

Stage	Time	What it needs
Rooting the S1 (Ch. 4)	~30 min, once ever	a Windows or macOS machine, micro-USB cable, Android Platform-Tools, the RoboMaster app
Raspberry Pi from blank (Ch. 5)	half a day — building Python 3.7 from source is the long part	Pi 4B, SD card (≥ 16 GB), card reader, keyboard + monitor for first boot, Ethernet cable
Network, camera, Carolus build (Ch. 6–9)	1–2 h, mostly compiling	USB-C cable (Pi $\leftrightarrow$ robot), the lab PC on the same network
A normal session afterwards (Ch. 2)	5 min	a charged robot battery, a powered beacon

Everything above except the rooting step runs from Linux. The rooting step is the one exception, and it is needed only once per robot.

Every command, parameter value, and code listing in this manual is the *current, corrected* version — the one that actually runs on this project today.

Chapter 2

Quick Start — A Typical Session

Runs on: the **lab PC** (the launcher and these commands), driving the **Raspberry Pi** over SSH. The robot must be powered on.

This chapter is for a machine that is already set up. If you are starting from bare hardware, skip it and follow Chapters 3–9 in order, then come back here — this is what every session after the first one looks like.

2.1 The five-minute version

1. **Power on the robot** — listen for **two chimes**, not one (Chapter 3). One chime means the root hack is not active and nothing below will work.
2. **Power on the Pi** with the button on the UPS HAT (Section 5.1 — the HAT’s port, not the Pi’s).
3. **Bring up the RNDIS link**, on the Pi. It does not survive a reboot, so this is needed every session (Chapter 6):

```
sudo ip link set eth1 up
sudo ip addr add 192.168.42.3/24 dev eth1
ping -c 4 192.168.42.2          # the robot -- expect 0% loss
```

4. **Start the launcher**, on the lab PC:

```
python3 shortcuts/carolus_launcher.py
```

5. **Click the tabs in order** (Chapter 11): T1, then T2, then T3. Each one waits for the previous.

That is a working stack: camera streaming, Carolus solving poses, the robot drivable from the keyboard.

2.2 Without the launcher, by hand

Useful when debugging, or when the launcher itself is the suspect. Three separate SSH shells to the Pi, in this order — each needs its own `ROS_MASTER_URI/ROS_IP`, they are per-shell:

```
# --- Shell 1, on the Pi: the ROS master ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
roscore

# --- Shell 2, on the Pi: the camera + SDK bridge (Chapter 6) ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/Python-3.7.17/env/bin/activate
python3.7 carolus_ws/src/robomaster_cam/scripts/rm_cam_beacon.py

# --- Shell 3, on the Pi: Carolus itself (Chapter 8) ---
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/carolus_ws/devel/setup.bash
roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false
```

2.3 Is it working? Three checks from the lab PC

Run these on the **lab PC**, not the Pi — that is the whole point: they confirm the nodes are visible across the network, not just locally.

```
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.100

rostopic hz /camera/color/image_raw    # expect ~16-30 Hz
rostopic hz /pose                       # ~13 Hz, ONLY with a beacon in view
roslaunch carolus_node testcarolus.launch # expect /ep_camera_beacon_follower
```

If `rostopic` finds nothing while the Pi says the node is running, the cause is almost always a missing `ROS_MASTER_URI/ROS_IP` in the shell that launched it — `rospy` then falls back to `localhost` and the node is invisible from anywhere else.

2.4 Shutting down

Close the launcher — it kills its own remote processes and verifies they died. By hand, on the Pi, the bracket in each pattern is not optional (see Chapter 11):

```
killall -f "[r]m_cam_beacon"
killall -f "[r]oscore"; killall -f "[r]osmaster"
pgrep -af "roscore|rosmaster|rm_cam_beacon"    # confirm nothing is left
```

Always confirm the bridge is dead before the next session. An orphaned `rm_cam_beacon.py` keeps the robot's single SDK connection open, and the next launch then hits the double-connection failure — motion commands stop working with no error shown (Chapter 7).

Chapter 3

Powering On the Robot

Runs on: the **robot itself** (physical handling). The Raspberry Pi and lab PC are not involved until Chapter 5.

3.1 Before you start

- The S1’s smart battery, charged. Charging is required before first use even if the battery shows some charge — this “wakes up” the battery.
- The S1 charger, plugged into mains (100–240 V, 50/60 Hz). A full charge takes approximately 1 h30.
- The battery only charges between 5 °C and 45 °C.
- Do not touch the motors, motor mounting plate, or the inside of the Mecanum wheels immediately after powering the robot off — they can be hot.
- Do not attempt to charge a battery that has dropped below 1 V per cell; this poses a fire risk.
- Use the official DJI charger only.

3.2 Power-on sequence

The battery compartment is behind the chassis’s rear armor panel, so the sequence is: open, insert, power on, close.

1. Press the rear armor release button to open the chassis’s rear cover.
2. Insert the smart battery into its compartment, making sure it is firmly seated — a poorly seated battery gives poor contact and can cause the robot to lose battery information.
3. Press and hold the power button (on the battery itself) for more than two seconds to power on.
4. Close the chassis’s rear armor.

The Raspberry Pi at the turret is a separate power domain from the robot chassis; there is no required power-on order between the two, and the USB/RNDIS cable connecting them is for the SDK link (Chapter 6), not for powering either device.

3.3 Confirming the robot is powered on

Expected light signals at power-on:

- **Gimbal (head) LED:** blinks cyan, counterclockwise.
- **Chassis LED:** solid cyan.
- **Motion controller LED:** blinks blue slowly — this means the motion controller is working normally.

3.4 Confirming the robot is rooted

This project assumes a rooted S1 exposing the RoboMaster EP SDK. Without the root hack (Chapter 4), SDK commands (chassis, gimbal, camera) will not respond, even though the robot otherwise powers on and behaves normally through the stock RoboMaster app.

Audible cue at boot: a correctly rooted S1 emits two chimes instead of the usual single chime. The hack persists across power cycles — it does not need to be reapplied every time the robot is powered on.

Concrete test, once the Pi is connected over RNDIS (Chapter 6): the camera/beacon node (`rm_cam_beacon.py`, Chapter 7) opens the one and only SDK connection this project uses:

```
1 self.ep = robot.Robot()  
2 self.ep.initialize(conn_type="rndis")
```

- **Success:** the node's log shows the connection succeeding, and the robot responds to `Twist` commands published on `/carolus/cmd_vel` (Chapter 7).
- **Failure:** the SDK connection fails or times out. First confirm the network link itself (ping 192.168.42.2 from the Pi, Chapter 6). If the network link is fine but the SDK connection still fails, redo the power-on and listen carefully for the double chime — a single chime means the root hack is not active.

Do not use any script other than `rm_cam_beacon.py` to test this. Two independent scripts each attempting `ep.initialize(conn_type="rndis")` at the same time silently blocks robot motion after a few commands, with no error surfaced — this failure mode is why this project uses exactly one script holding the SDK connection at any time (see Chapter 7).

3.5 Which camera is mounted

This project uses the S1's own internal camera, not an external Intel RealSense D455. Physically: a D455 is an external rectangular camera housing mounted in addition to the robot's own head; if there is no such external camera, the internal camera is in use. In software, once `rm_cam_beacon.py` is running (Chapter 7), the internal camera feed appears on `/camera/color/image_raw` immediately, with no separate D455 driver (`rs_camera.launch`) needed.

Chapter 4

Rooting the S1

Runs on: a **Windows or macOS computer** connected to the robot by micro-USB. Not the Raspberry Pi, not the Ubuntu lab PC — the RoboMaster app required for this step does not run on Linux.

If Chapter 3’s double-chime check already confirmed the robot is rooted, this chapter is informational only — do not re-run it.

4.1 What the hack does

DJI’s RoboMaster app exposes a limited, sandboxed Python scripting environment (“the Lab”). The S1’s Python subsystem runs with root privileges internally, but DJI restricts which modules a Lab script can import via a whitelist attached to the `__builtins__` object. Several of the whitelisted modules (needed for legitimate scripting features) still hold a reference to the real, unrestricted `__builtins__` internally. That reference can be recovered and used to import `subprocess`, which can then invoke a shell script already present on the robot’s filesystem (`adb_en.sh`) to enable `adb` and open a root shell over USB.

Once a root shell is available, a patch is pushed to the robot’s filesystem that installs the RoboMaster EP SDK’s full command set (the EP is DJI’s higher-end sibling to the S1; underneath, the two share nearly identical firmware) onto the S1, which DJI has never officially released to S1 owners.

Warning, carried over from the hack’s own documentation, to be taken literally: this has the potential to break the robot. It did not break this project’s unit, but there is no guarantee for any given unit.

4.2 Step-by-step procedure

4.2.1 What you need

- A computer running Windows or macOS (the RoboMaster app requires one of these to configure the S1; the Raspberry Pi is not involved in this chapter).
- Android SDK Platform-Tools (`adb.exe` and friends) — unzip anywhere, remember the path.
- A micro-USB cable.
- The RoboMaster app, with a DJI account.

4.2.2 Procedure

1. Connect the computer to the S1's Intelligent Controller Micro-USB port with the micro-USB cable. Ensure no other Android device is connected over USB during this procedure.
2. Power on the robot (Chapter 3) and connect the RoboMaster app to it via its own Wi-Fi hotspot (visible on top of the gimbal, with a password shown alongside it).
3. In the app: **Lab** → **DIY Programming** → select **Python** on the top bar → **New** to create a new program.
4. Paste the following into the program editor exactly. This is the current, working root script for this hack version — do not use an older variant that imports `'subprocess'` as a literal string or reads from `rm_log` instead of `rm_define`; a firmware update closed that specific path.

```
1 def root_me(module):
2     __import__=rm_define.__dict__['__builtins__']['__import__']
3     return __import__(module, globals(), locals(), [], 0)
4
5 builtins=root_me('builtins')
6 x=root_me('sub'+ 'process')
7 proc=x.Popen('/system/bin/adb_en.sh', shell=True, executable='/system/bin/
8     sh',
9             stdout=x.PIPE, stderr=x.PIPE)
```

5. Run the program from within the Lab. A correct run shows no compilation errors and the console prints **Execution Complete**. Do not close the RoboMaster app.
6. On the computer, open a terminal in the folder containing the Android Platform-Tools and run:

```
adb devices
```

The S1 should appear in the device list.

7. Confirm root access:

```
adb shell
```

8. With the root shell open, from a second terminal, in the folder containing the hack's distribution files (`dji_scratch`, `dji.json`, `dji_hdvt_uav`, `dji_sys`, `patch.sh`), push the patch:

```
adb shell rm -rf /data/dji_scratch
adb push dji_scratch /data/.
adb push dji.json /data/.
adb push dji_hdvt_uav /data/.
adb push dji_sys /data/.
adb shell chmod 755 /data/dji_hdvt_uav
adb shell chmod 755 /data/dji_sys
adb push patch.sh /data/.
```

9. Restart the S1 fully (hold the power button until it turns off, then hold it again to turn back on). On boot, listen for **two chimes instead of one** — this confirms the hack is active (Chapter 3's check).
10. Close the RoboMaster app and power off the robot; the setup for this chapter is complete.

4.3 A pitfall to know before rooting

Starting with hack version 0.0.3, `adb` is re-enabled by default in the patch, which **disables RNDIS support** — the exact link this project’s Pi↔robot connection depends on (Chapter 6). If RNDIS stops working after rooting, comment out the `adb` line in `patch.sh` before pushing the patch, or re-push a corrected `patch.sh` and restart the robot again.

Chapter 5

Setting Up the Raspberry Pi

Runs on: the **Raspberry Pi 4B** (mounted on the robot), with a keyboard/monitor for first boot or over SSH from the lab PC afterwards.

This chapter takes a blank Raspberry Pi 4B to a working Ubuntu 20.04 installation with Python 3.7, the RoboMaster SDK, ROS Noetic, and SSH access. If the Pi is already set up, skip to Chapter 6.

5.1 Powering the Pi — two USB-C ports, only one of them right

Read this before plugging anything in. The Pi carries a UPS HAT (a green PCB underneath it, with its own battery and a physical ON/OFF button on the back). That means **two different USB-C ports**, which look interchangeable and are not:

- **The HAT’s USB-C port** (on the green PCB underneath) — **this is the one to use.** It charges the internal battery, a blue “Charge” LED lights up, the ON/OFF button works, and the Pi then runs standalone on battery.
- **The Pi’s own POWER IN port** (on the Pi board itself, to the left of the HDMI connectors) — powers the Pi directly. It boots, but the HAT’s ON/OFF button is ignored and the Pi is tethered to the cable. Not for normal sessions.

If the Pi will not turn on: check the cable is on the HAT port and not the Pi’s POWER IN, and that the charge LED is blue. This is the single most common physical blocker on this setup.

Normal routine: plug the charger into the HAT port the evening before a session. The next day the Pi runs on its own battery, started and stopped with the physical button — no cable, no tether.

5.2 Flashing Ubuntu Server

1. Download the Ubuntu Server ARM64 image from <https://cdimage.ubuntu.com/releases/focal/release/ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz>.
2. Decompress it:

```
unxz ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz
```

3. Identify the SD card device (do this carefully — writing to the wrong device destroys data on it):

```
lsblk
```

4. Flash it:

```
sudo dd if=ubuntu-20.04.5-preinstalled-server-arm64+raspi.img of=/dev/sdX  
bs=4M status=progress conv=fsync
```

5. Boot the Pi with the card inserted, plus keyboard, mouse, display, and an Ethernet cable for first boot. Default credentials: `ubuntu/ubuntu` (you will be asked to change the password on first login).

5.3 Desktop environment (optional)

Only needed if you want a graphical session on the Pi itself (not required for this project's normal operation, which runs headless):

```
sudo apt update  
sudo apt install ubuntu-desktop  
sudo reboot
```

5.4 Enabling headless SSH access

If SSH was not already enabled at flash time (Raspberry Pi Imager's advanced options support this directly), enable it on first boot:

```
sudo systemctl enable ssh  
sudo systemctl start ssh  
sudo systemctl status ssh # optional: confirm it's running
```

Find the Pi's IP address on the local network:

```
hostname -I
```

Connect from the client machine, first with a password, then switch to key-based login:

```
# First connection (password):  
ssh <username>@<pi_ip>  
  
# On the client: generate a key  
ssh-keygen -t ed25519 -C "client-key"  
  
# Copy it to the Pi  
ssh-copy-id <username>@<pi_ip>  
  
# Test key-based login (no password prompt expected)  
ssh <username>@<pi_ip>
```

A convenient alias, in `~/.ssh/config` on the client:

```
1 Host raspberry  
2   HostName <pi_ip>
```

```
3 User <username>
4 IdentityFile ~/.ssh/id_ed25519
```

Note the exact directive name: `IdentityFile`, not `IdentifyFile` — a common typo that OpenSSH silently rejects the config over.

Connect with `ssh raspberry` from then on.

Optional hardening, once key-based login is confirmed working:

```
# Firewall
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw enable

# Disable password authentication (only after keys work!)
sudo nano /etc/ssh/sshd_config
# Ensure: PasswordAuthentication no
#         PubkeyAuthentication yes
sudo systemctl restart ssh
```

5.5 Building Python 3.7 from source

The RoboMaster SDK requires Python 3.7 specifically; Ubuntu 20.04 ships a newer default Python, so 3.7 is built from source into a separate location rather than replacing the system Python.

```
sudo apt update
sudo apt install -y build-essential libssl-dev zlib1g-dev \
    libreadline-dev libsqlite3-dev libbz2-dev libexpat1-dev \
    liblzma-dev tk-dev libffi-dev uuid-dev

cd ~/Python-3.7.17/
./configure --enable-optimizations
make -j4
sudo make altinstall
```

Using `altinstall` (not `install`) is deliberate: it installs as `python3.7` alongside the system Python, without overwriting it.

Confirm the SSL module built correctly (a common source of a broken venv later if skipped):

```
python3.7 -c "import ssl; print(ssl.OPENSSL_VERSION)"
```

5.6 Virtual environment and the RoboMaster SDK

Still in `~/Python-3.7.17/` from the previous section — the venv is created there, not at the home directory root, and every later reference to it in this manual (Chapter 7) uses the full path for exactly that reason:

```
python3.7 -m venv env
source env/bin/activate
pip install --upgrade pip
pip install robomaster numpy
pip install "opencv-python==4.5.5.64"
```

The pinned OpenCV version is a historically validated compatible version for this SDK/Python combination on this hardware; it is not a hard requirement enforced elsewhere in this manual, and later versions have since been used successfully on the lab PC (Chapter 9) without issue.

The lab PC and the Pi do not run the same robomaster SDK version, and this matters. Measured directly from each machine’s installed package, not from a requirements file: **0.1.1.68** on the lab PC, **0.1.1.62** on the Pi. The Pi’s is the one that actually drives the robot, since `rm_cam_beacon.py` runs there. For any question about SDK behaviour — a call’s return type, a parameter’s sign convention, a value range — read the source installed *on the Pi*, never the lab PC’s copy and never online documentation for an unstated version. Neither version’s own docstring for `chassis.drive_speed/chassis.move` documents the sign of the `z` parameter, only its range — any sign convention this project relies on comes from hardware measurement, not the SDK.

5.7 Installing ROS Noetic

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu \
$(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
sudo apt install curl
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \
    | sudo apt-key add -
sudo apt update

# Full desktop version:
sudo apt install ros-noetic-desktop-full
# --- or, if the Pi struggles with the desktop version's resource use ---
sudo apt install ros-noetic-ros-base

echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc

sudo apt install python3-rosdep python3-rosinstall \
    python3-rosinstall-generator python3-wstool build-essential
sudo rosdep init
rosdep update --include-eol-distros
```

The `-include-eol-distros` flag is not optional. ROS Noetic reached end-of-life in 2025, and a plain `rosdep update` silently skips it (Skip end-of-life distro "noetic", easy to miss in the scroll of output) — after which `rosdep` has no package-name mapping for *any* ROS package on this distro, including ones as basic as `roscpp`, and every `rosdep resolve/ rosdep install` against this workspace fails with `no rosdep rule for '...'`. Confirmed directly: the same `rosdep update` without the flag, then with it, on this lab PC.

On this project’s Pi, the lighter `ros-noetic-ros-base` is what actually runs day to day — the Pi only needs to run `rm_cam_beacon.py` (Chapter 7), not a full desktop ROS toolchain.

`ros-noetic-ros-base` does not include the perception stack this project’s own packages need (`image_transport`, `cv_bridge`, `image_geometry`) — fixing that needs the workspace to already be cloned, so that step is in Section 9.3, right before the build itself rather than here.

5.8 Getting this project’s code onto the Pi

The Pi needs this project’s own workspace — specifically the clone’s `carolus_ws/src/robomaster_cam/` subdirectory, which holds `rm_cam_beacon.py` (Chapter 7) and its companion scripts. Clone the project’s repository directly onto the Pi:

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git
```

This creates a `robotmaster/` directory whose `carolus_ws/` subdirectory is the actual catkin workspace — `robotmaster/carolus_ws/src/robomaster_cam/` is the path this manual means whenever it says `carolus_ws/src/robomaster_cam/`. Renaming `robotmaster/` to anything else, including `carolus_ws`, is fine; just keep referring to its `carolus_ws/` subdirectory as `carolus_ws/`, not the clone root itself.

This uses an SSH key registered against *your own* GitHub account (Chapter 9 covers generating and registering one). The repository is public, so no collaborator access is needed to read it — this only matters the moment you are given write access. Either way, **do not paste a shared personal access token into the clone URL**: it grants whoever reads it the full rights of the account that made it, and cannot be traced back to an individual. If SSH is not set up, an HTTPS clone works with no credentials at all:

```
git clone https://github.com/Arracheur2Bonnet/robotmaster.git
```

No build step is needed on the Pi side for `robomaster_cam` — it is a pure-Python package, run directly (Chapter 7).

5.9 A pitfall to expect at first launch

The RNDIS network interface (Chapter 6) does not come up automatically at Pi boot, even once everything above is configured correctly. This is expected — Chapter 6 gives the exact two commands to run at the start of every session.

Chapter 6

Network Setup (RNDIS)

Runs on: the **Raspberry Pi** for the interface commands; one step at the end runs on the **lab PC** and is marked as such.

The robot exposes an RNDIS virtual Ethernet interface over the same USB cable used for the SDK connection. The robot holds a fixed address, 192.168.42.2; the Raspberry Pi must be configured on the same subnet.

6.1 Bringing up the link

Run these two commands at the start of every session, on the Pi. The interface does not persist across Pi reboots — this is expected behavior on this hardware, not a fault, and there is no step elsewhere in this manual that avoids needing it.

```
sudo ip link set eth1 up
sudo ip addr add 192.168.42.3/24 dev eth1
```

If the interface is not named `eth1` on your system, identify it first:

```
ip -br addr
# look for eth1 / usb0 / enx<MAC> -- whichever one is meant to carry
192.168.42.x
```

Never assign 192.168.42.2 to the Pi — that address belongs to the robot.

6.1.1 If you intend to automate this over SSH

Typing the two commands by hand prompts for a password, which is fine interactively. **Any script that brings the interface up over SSH needs passwordless sudo on the Pi**, otherwise it hangs silently waiting for a prompt it cannot display — the process simply appears frozen, with no error. Check before automating anything:

```
ssh <user>@<pi> 'sudo -n true && echo "passwordless sudo OK"'
```

If it fails, grant it with `visudo` on the Pi (scope it to the two `ip` commands rather than granting blanket rights).

6.2 Verifying the link

```
ping -c 4 192.168.42.2
```

Expect 0% packet loss, sub-millisecond round-trip time once the interface is correctly up.

6.3 A quick automated alternative

Rather than typing the two commands above by hand every session, they can be scripted:

```
sudo ip link set eth1 up && sudo dhclient eth1
```

This project's own operator tooling (not covered in this manual) runs the equivalent of the first two-command form automatically as part of its first launch step, over SSH from the lab PC.

6.4 Lab PC hostname resolution

If any ROS tool on the lab PC (e.g. `rqt_image_view`) shows a black image or errors with something like `Couldn't find AF_INET address for [ubuntu]`, add the Pi's hostname to `/etc/hosts` on the lab PC:

```
echo "192.168.0.103_ubuntu" | sudo tee -a /etc/hosts
```

Unlike the RNDIS interface on the Pi, this fix is permanent on the lab PC and does not need to be repeated.

6.5 Summary of addresses used in this manual

Device	Address
Robot (over RNDIS)	192.168.42.2
Raspberry Pi (over RNDIS, to the robot)	192.168.42.3
Raspberry Pi (on the local network, to the lab PC)	192.168.0.103
Lab PC (on the local network, to the Pi)	192.168.0.100

The last two are DHCP-assigned local-network addresses and may differ on a different network; find them with `hostname -I` run on each machine (Chapter 5). Both are used again in Chapter 9 as `ROS_IP` when launching a node from that machine.

Checkpoint — do not continue until this passes. From the Pi, `ping -c4 192.168.42.2` returns 0% loss. From the lab PC, `ping -c4 192.168.0.103` reaches the Pi. If either fails, nothing in the following chapters can work, and the failure will look like a broken camera or a broken node rather than a broken link.

Chapter 7

The Camera and Robot Bridge

Runs on: the **Raspberry Pi** — this is the only machine that ever connects to the robot SDK. The verification commands at the end run from the **lab PC**.

7.1 Design: a single SDK connection

The Pi runs exactly one script that ever connects to the robot’s SDK: `rm_cam_beacon.py` (in `carolus_ws/src/robomaster_cam`). It is not just a camera capture script — it is this project’s single bridge to the entire SDK: camera stream, chassis drive, gimbal control, odometry, and the IMU subscription all go through this one script and its one connection.

This is a deliberate design choice, not an incidental one. Two independent scripts each holding their own `ep.initialize(conn_type="rndis")` connection at the same time silently blocks robot motion commands after a few calls, with no error surfaced. **Never run any other script that calls `robot.Robot().initialize(...)` while `rm_cam_beacon.py` is running.**

7.2 Starting the bridge

On the Pi, with the RNDIS link up (Chapter 6) and the Python 3.7 virtual environment active (Chapter 5). The ROS master runs on the Pi itself (Section 9.1), so `ROS_MASTER_URI` and `ROS_IP` must both point at the Pi’s own address on the lab network, `192.168.0.103` (Chapter 6) — without them `rospy` falls back to `localhost` and the node is invisible to every other machine:

```
source /opt/ros/noetic/setup.bash
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/Python-3.7.17/env/bin/activate
python3.7 carolus_ws/src/robomaster_cam/scripts/rm_cam_beacon.py
```

Source ROS *before* activating the venv, in the same shell: the venv only prepends its own `bin/` to `PYTHONPATH`, it does not clear the ROS environment variables sourced ahead of it.

In normal operation this is launched automatically by this project’s own operator tooling (the Tkinter launcher in `shortcuts/`, documented in `shortcuts/README.md` in the repository), over SSH from the lab PC — it is not usually started by hand.

7.3 What it connects to

```

1 self.ep = robot.Robot()
2 self.ep.initialize(conn_type="rdis")
3 self.chassis = self.ep.chassis
4 self.gimbal = self.ep.gimbal
5 self.cam = self.ep.camera
6 self.cam.start_video_stream(display=False)

```

From there it subscribes to the SDK's own push-data channels (`sub_position`, `sub_attitude`, `sub_velocity`, `sub_imu`, `sub_angle` for the gimbal), converts each callback into a ROS message, and publishes:

Topic	Content
<code>/camera/color/image_raw</code>	Camera frames (<code>sensor_msgs/Image</code>)
<code>/odom</code>	Wheel odometry, republished from the SDK
<code>/imu</code>	Raw IMU, 50 Hz requested — units: see warning
<code>/carolus/gimbal_yaw_rel</code>	Gimbal yaw relative to the chassis
<code>/carolus/gimbal_yaw_ground</code>	Gimbal yaw in the world frame (absolute)

/imu units are g , not m/s^2 (measured 2026-08-04). At rest on a flat floor, this topic reports `linear_acceleration.z` = -1.00664. The SDK returns accelerations in g , and `rm_cam_beacon.py` republishes them unconverted — whereas `sensor_msgs/Imu` mandates m/s^2 . Any consumer of this topic therefore sees accelerations $9.81\times$ too small under a message contract that says otherwise, and neither Kalibr nor MINS will detect it. The sign is also suspect: a REP-103 z -up frame at rest should read +9.81, not $-1.0g$. Gyroscope units and axis signs are likewise unconfirmed — a reading at rest cannot distinguish rad/s from $^\circ/s$ when the true rate is zero.

Before using this topic, verify on your own robot: gravity should read ≈ 9.81 on exactly one axis at rest, and a known slow rotation should give the expected sign and magnitude on the corresponding gyro axis. Fix scale, sign and gyro units together — correcting the scale alone yields a topic that looks right while its frame convention is still unverified.

Two further constraints: this SDK accepts only 1, 5, 10, 20 and 50 Hz for this subscription, so 50 Hz is the ceiling — below the 200–500 Hz that camera-IMU calibration tools such as Kalibr (Chapter 8) normally expect. The frequency is exposed as the `RM_IMU_FREQ` environment variable so the other legal values can be tried without editing the script.

If /imu is silent, check the publisher creation order before suspecting the robot.

Every ROS publisher written to from an SDK callback must be created *before* the corresponding `sub_*` subscription. The robot begins pushing data the moment the subscription is registered, and on the SDK build used here the callback runs directly in the DDS dispatcher thread: an `AttributeError` on a not-yet-created publisher kills that thread, after which *no* telemetry callback ever runs again. The symptom is indistinguishable from a robot that sends nothing — the topic is advertised, `sub_imu` returns `True`, and ROS logs nothing. This cost a full session to diagnose.

and subscribes to a small set of control topics used by this project's own higher-level tooling (not covered in this manual): `/pose` (Carolus's own output, consumed here to drive autonomous behavior), `/carolus/mode`, `/carolus/cmd_vel`, `/carolus/gimbal_vel`, `/carolus/gimbal_lock`, `/carolus/gimbal_lock_per` and `/carolus/gimbal_recenter`.

7.4 Verifying it is running

From the lab PC, with the ROS master reachable:

```
rostopic list | grep camera
rostopic hz /camera/color/image_raw
rostopic echo -n1 /odom
rostopic echo -n1 /imu
```

Measured on the Raspberry Pi with the robot connected (2026-08-04): `/camera/color/image_raw` at **30.1 Hz**, `/odom` at **15.4 Hz**, `/imu` at **47.3 Hz** against a 50 Hz subscription. Expect at least one non-empty message on `/odom` and `/imu`; if `/imu` hangs, read the second warning above before concluding anything about the robot. If the camera topic is silent, confirm the RNDIS link (Chapter 6) before suspecting the script itself — a dead network link and a dead camera stream look identical from this check alone.

Two further diagnostic lines to expect in this node’s own log, neither of which needs any setup: `[MODE]` appears once at startup, reading the robot’s current mode, forcing `FREE`, then reading it back and warning on a mismatch; `[DUTY] beacon in view NN%` appears every 30 s, the fraction of that window with a beacon actually detected — 0% with no beacon in view is correct, not a fault.

Checkpoint — do not continue until this passes. From the lab PC (not the Pi — that is the point of the check): `rostopic hz /camera/color/image_raw` reports a rate, and `rostopic echo -n1 /odom` returns one message. If the topics exist on the Pi but not from the PC, the bridge was launched without `ROS_MASTER_URI/ROS_IP` — see the launch command above.

7.5 Gimbal yaw: two signals, two different meanings

`/carolus/gimbal_yaw_rel` and `/carolus/gimbal_yaw_ground` are not interchangeable, and confusing them is a documented source of bugs in this project’s history:

- **`gimbal_yaw_ground`** is the gimbal’s heading in the world frame. The S1’s gimbal is yaw-stabilized: when the chassis rotates without an explicit gimbal command, the gimbal holds its absolute heading. This value stays essentially constant across a chassis rotation.
- **`gimbal_yaw_rel`** is the gimbal’s angle *relative to the chassis*. Because the gimbal holds its world heading while the chassis turns underneath it, this value *absorbs* the chassis’s rotation — it is not a stable “how far off is the beacon” signal across a chassis maneuver, it is a signal that actively tracks how much the chassis has turned since the gimbal last held still relative to it.

Use `gimbal_yaw_ground` when you need to know the camera’s absolute heading in the world. Use `gimbal_yaw_rel` only for what it actually is: the angle between the gimbal and the chassis at this instant, not a proxy for beacon bearing across a chassis rotation.

7.6 The internal camera vs. an external D455

This robot uses its own internal camera exclusively. If, when following any inherited reference material, you encounter instructions to launch `rs_camera.launch` or install `librealsense`, that material concerns a different camera than the one on this robot and does not apply here — see the calibration discussion in Chapter 8 for where that material remains genuinely useful (the calibration *technique*, not the camera driver).

Chapter 8

Camera Calibration

Runs on: the lab PC (or any machine with MATLAB / a Kalibr install). Images and rosbags come from the robot camera over the network.

8.1 Current values in use

The intrinsics and distortion coefficients currently loaded by `testcarolus.launch` (Chapter 9) are:

Parameter	Value
f_x	546.1957
f_y	547.0838
c_x	575.6041
c_y	372.1876
Distortion (plumb_bob: k_1, k_2, p_1, p_2)	-0.1479, 0.1452, -0.0023, 0.0025

These come from `robomaster_cam/config/camera_info.yaml` (`camera_name: robomaster_s1`, 1280×720), validated pragmatically — Carolus produces a stable, coherent pose with these values in practice. They are **not** the result of a formal metric calibration campaign (checkerboard/AprilGrid target, dedicated calibration tool run to convergence with a reported reprojection error). If you are setting up a new unit, or want a rigorous calibration, follow the procedure below rather than reusing these values — they are specific to this project’s camera and are not guaranteed to transfer to a different physical unit.

8.2 Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox

This procedure has not yet been run against this project’s own camera — the values in the previous section are what is actually loaded today. It is included here as the recommended path for whoever runs the next calibration, based on a direct comparison against Kalibr on a comparable camera (Section 8.4).

1. Obtain a checkerboard target of known square size. **On this project there is nothing to print: the checkerboard is on the back of the beacon** (Section 9.9.2) — turn the beacon around and use that. If you are working with a different beacon, print one; 25 mm squares is a proven working size for this class of camera, and is what both prior Carolus integrations used.

2. Capture a set of images of the board with the robot’s camera (Chapter 7) using `shortcuts/capture_checkerbo` (run on the lab PC; it subscribes to `/camera/color/image_raw` over the network and saves a PNG on each keypress): at least 10 images, 15 or more for better accuracy. The board should occupy roughly 20–25% of the frame area, and be tilted more than 45° from the camera’s optical axis in each shot to avoid detection errors. Capture from a wide variety of positions and orientations across the field of view.
3. In MATLAB: open the **Camera Calibration app** (Apps tab), add the captured images, select the checkerboard pattern with the correct square size, and enable **tangential distortion** and **three radial distortion coefficients** (this camera class shows moderate wide-angle distortion that the default two-coefficient model under-fits). **Requires the Computer Vision Toolbox** — checked directly (2026-08-11, this lab PC’s desktop MATLAB R2025b): the app and its underlying functions (`cameraCalibrator`, `estimateCameraParameters`, `detectCheckerboardPoints`) are not present by default even with a valid MATLAB licence covering the toolbox; install it once via Home → Add-Ons → Get Add-Ons if the app does not appear. MATLAB Online comes with every toolbox already provisioned and needs no such install.
4. Run the calibration. Export the resulting coefficients to the MATLAB workspace and generate the calibration script — this script contains the intrinsic matrix, distortion coefficients, and reprojection errors in a form that can be transcribed directly into `testcarolus.launch`’s `fx/fy/cx/cy/distortion` parameters (Chapter 9).

8.3 Known limitation: no confirmed auto-exposure control

Two independent prior integrations of Carolus on other robots (a Leo Rover and a Limo, both using standard UVC/V4L2-accessible cameras) report the same failure mode: camera auto-exposure drifting while the robot moves biases every pose Carolus reports, because the beacon-detection threshold (Section 9.9) is tuned for one brightness level. One of the two integrations spent real effort misattributing the resulting trajectory distortion to lens/fisheye distortion before finding the actual cause. Both fixed it by disabling auto-exposure and locking a fixed exposure time at the OS/driver level.

This project’s camera does not offer a confirmed equivalent. The `robomaster` Python package this project’s code actually calls (`rm_cam_beacon.py`) exposes no exposure control of any kind — checked directly against the installed package source, not assumed. Unlike the two prior integrations, this camera is not a plain V4L2/UVC device the Pi can address directly; it is delivered through DJI’s own SDK protocol. The rooted SDK’s own protocol tables do list one lower-level command (`camera.exposure`, four discrete bias levels, no way to read the current value back), reachable only through the separate on-robot Scratch API layer, not confirmed reachable from the SDK connection this project’s code holds — and even if it were, it biases auto-exposure rather than disabling it, which is a different capability from what fixed the problem for the two prior integrations.

Practical consequence. Auto-exposure drift while the robot moves is a plausible, currently unquantified contributor to beacon-detection instability on this platform. It has not been measured here. If Carolus’s pose grows visibly less reliable during motion than at rest, this is a candidate cause worth checking before assuming a geometric or software explanation — the same trap one of the two prior integrations fell into.

8.4 Alternative / complementary method: Kalibr

Kalibr (ETH Zürich) is the standard tool if a camera–IMU joint calibration is needed (required for any future MINS/visual-inertial integration, not needed for Carolus’s own pose estimation, which does not

use the IMU).

Not executed here. The camera–IMU half of this procedure has never been run on this robot. The camera-only calibration below is unaffected and is the part that matters for Carolus. Two constraints to know before planning a camera–IMU session on this platform: the SDK caps the IMU subscription at 50 Hz, well below the 200–500 Hz Kalibr expects for a well-conditioned solution; and `/imu` publishes accelerations in g rather than the m/s^2 the message type mandates (Chapter 7), with gyroscope units and axis signs still unconfirmed. Correct the units and verify the frame first — Kalibr will not detect either error, and every noise parameter it produces would be wrong by the same silent factor.

8.4.1 Target

A 6×6 AprilGrid, tag size 0.089 m, tag spacing $0.3 \times$ tag size (an $\sim 0.8 \times 0.8$ m printed board). Measure the actual printed square size and use the measured value, not the nominal one — even a $\sim 1\%$ sizing error biases the recovered scale.

```
1 target_type: 'aprilgrid'
2 tagCols: 6
3 tagRows: 6
4 tagSize: 0.089          # metres -- MEASURE your own printed board
5 tagSpacing: 0.3         # ratio of gap to tag size
6 codeOffset: 0
```

8.4.2 Installing Kalibr

```
cd ~/camera_ws/src
git clone https://github.com/ethz-asl/kalibr.git
cd ~/camera_ws
rosdep install --from-paths src --ignore-src -y
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-
generator \
    build-essential python3-catkin-tools
sudo apt-get install libv4l-dev doxygen python3-igraph
sudo apt install python3-pip
pip install wxPython      # can take a long time -- be patient
catkin_make -DCMAKE_BUILD_TYPE=Release
source ~/camera_ws/devel/setup.bash
```

Do not separately clone `numpy_eigen` or `Schweizer-Messer` — Kalibr bundles its own copies, and cloning them independently into `src/` produces duplicate-package build errors. If you hit `ModuleNotFoundError: No module named 'sm'` after building, verify with:

```
python3 -c "import sm; print(sm.__file__)"
```

and if it fails, remove any independently-cloned copy of **Schweizer-Messer**, then rebuild:

```
rm -rf build/ devel/ install/
catkin_make -DCMAKE_BUILD_TYPE=Release
```

8.4.3 Capturing data and running the calibration

Record a rosbag while moving the camera (or the robot itself, for a joint camera–IMU calibration) through the AprilGrid’s field of view for 60–90 seconds, exciting all six degrees of freedom (translate

near/far/side to side, rotate about all three axes, not just yaw).

```
rosbag record -O calib_data.bag /camera/color/image_raw
```

Run the calibration:

```
export KALIBR_MANUAL_FOCAL_LENGTH_INIT=1
kalibr_calibrate_cameras \
  --bag calib_data.bag --target target.yaml \
  --models pinhole-radtan \
  --topics /camera/color/image_raw \
  --approx-sync 0.01
```

If Kalibr reports **Initialization of focal length failed**, provide a manual seed value (600 is a reasonable starting guess for a 640×480 -class image; scale proportionally for other resolutions).

8.4.4 Reading the result

Kalibr writes three files: a `.txt` summary, a `.pdf` of reprojection-error plots, and a `.yaml` with the calibration parameters. **Target: reprojection error under 0.2–0.5 pixels.** In practice, a completed run can land a little outside that target (observed: 0.59–0.67 px std on a comparable camera) without necessarily being unusable — treat the target as a goal to aim for, not a hard pass/fail gate, and judge the calibration by whether downstream pose estimates are stable and physically plausible.

8.5 Camera–IMU time offset

If a Kalibr camera–IMU joint calibration is run, its output `timeshift_cam_imu` is in **seconds**, with the convention $t_{\text{imu}} = t_{\text{cam}} + \text{shift}$. This matches OpenVINS’s own internal convention exactly — the value can be used directly with no sign flip if it is later consumed by an OpenVINS-based filter.

Chapter 9

Building and Launching Carolus

Runs on: the Raspberry Pi 4B for actually running Carolus — this is where the node executes, full stop, not one of two equally valid options. The **lab PC** build is documented too, but only as a development environment for editing and compiling the C++ before deploying it to the Pi; Section 9.1 gives the measurements behind why operation and development are split this way.

Carolus (the ROS package name is `carolus_astrobee`) has no SDK connection of its own — it consumes `/camera/color/image_raw` (published by `rm_cam_beacon.py`, Chapter 7) and publishes a pose on `/pose`, so it only needs to reach the ROS master, not the robot. That is *why the lab PC can build and run this code at all* despite never touching the robot directly — not a reason to operate it there, which the next section settles.

9.1 Where Carolus runs

The Pi runs Carolus, always — autonomy, not speed, is the decisive argument. A robot whose pose estimator lives on another machine stops working the moment the link does; that is a property of the architecture, not a number a future optimisation could fix. The 2–3× camera input rate and the absence of build workarounds (below) are real, but secondary. **The lab PC builds and edits the same code but never operates the robot with it** — a development convenience, kept because compiling and iterating C++ is faster on a workstation than over SSH and the operator GUI already runs there.

9.1.1 The two machines, side by side

	Raspberry Pi 4B (where Carolus runs)	Lab PC (development only)
Robot autonomy	Complete. No network link is needed to estimate pose.	None. Every pose depends on a live link; the robot cannot leave the PC's network reach.
Camera input rate (measured)	30.1 Hz — frames never leave the machine that captured them.	10–15 Hz — every frame is an <i>uncompressed</i> <code>sensor_msgs/Image</code> at 1280×720 (≈ 2.76 MB) crossing the network first.
Build complexity	Ubuntu 20.04 = ROS Noetic's officially supported target. Standard <code>catkin_make</code> , no workarounds (Section 9.5).	On Ubuntu 22.04: four build-time fixes plus one runtime fix (Section 9.6). On 20.04, identical to the Pi.
Development comfort	Editing and rebuilding C++ over SSH is slower.	Better. A workstation compiles faster and edits more comfortably; GUI tooling runs here.
Visual-inertial work (MIN-S/OpenVINS)	Inside the upstream tested set (Ubuntu 18/20, Melodic/-Noetic).	On 22.04, outside it.

9.1.2 Measured, 2026-08-04 — and what the numbers do *not* say

Same robot, same beacon at 1.00 m, same afternoon — Carolus run once on each machine and `/pose` measured both times:

	Carolus on the lab PC	Carolus on the Pi
<code>/pose</code> , beacon in view	2.19 Hz	13.04 Hz
<code>/postprocessed/image</code>	—	14.35 Hz
<code>/camera</code> at the Pi's publisher	30.05 Hz	16.52 Hz
Pi CPU	52–61% idle	12.3% idle , load 5.37/4

A factor of 5.95, from an uncompressed 1280×720 `sensor_msgs/Image` (≈ 2.76 MB) crossing the network on the PC versus never leaving the machine on the Pi. The other rows say where the limit moved to: `/pose` 13.0 Hz, `/postprocessed` 14.4 Hz and the camera at 16.5 Hz are nearly the same number, and the camera itself drops from 30.05 Hz to 16.52 Hz once Carolus runs beside it, with the Pi at 12.3% idle — **the bottleneck on the Pi is now CPU**, camera bridge and Carolus competing for it. If more pose rate is needed, look there: compressed frames, or a lower capture resolution.

Verify which machine you are measuring. Both numbers above are /pose rates with the same beacon, and the only difference is where the node ran. It is easy to get this wrong: the ROS master lives on the Pi in both cases, so a Carolus running on the workstation still looks Pi-side from the topic list. Confirm with `pgrep -f carolus_astrobees on the Pi` before attributing a rate to it.

Two older figures circulate and neither is a pose rate: a ~ 24 frames/s figure for the Pi was counted **with no beacon in view**, so no P4P solve ran on any frame, and a ~ 2.5 frames/s figure for the lab PC used a different counting method again. Use the table above, not those.

The Pi already had `libcere-dev 1.14.0-4ubuntu1.1`, `libeigen3-dev 3.3.7-2` and `libopencv-dev 4.2.0+dfsg-5` installed; `catkin_make` compiled and linked directly on `aarch64`. Available memory (7.6 GB) was never a constraint.

9.1.3 A note on visual-inertial navigation

MINS has been run on this Raspberry Pi, both on its own simulation and on data recorded from this robot. It builds on `aarch64` and produces a *consistent* estimate on the simulation (RMSE 0.113° / 0.082 m , NEES 0.9 and 1.4), with no divergence, at **$0.3\text{--}0.4\times$ real time** — under a sensor load far heavier than this robot’s (IMU at 200 Hz, *two* cameras, and a LIDAR).

On a 60 s recording of *this* robot’s `/imu` and `/camera/color/image_raw`, it processed the whole bag at **$0.7\times$ real time**: CAM 118 ms, IMU 3 ms, 122 ms of compute per 88 ms of data. So “**MINS does not run on a Raspberry Pi**” is not correct — it runs, on this robot’s own data. What is correct is that **it does not reach real time here**, even with one camera, no LIDAR and a 50 Hz IMU, and that essentially all of the cost is the camera update. The $0.7\times$ is also optimistic: `rosbag record` throttled the sources to 16 Hz of camera against 30 Hz live.

Two practical points if you repeat this. MINS’s live-subscribe entry point is **ROS 2 only** — in `run_subscribe.cpp` the `ROS_AVAILABLE == 1` branch contains two `#include` lines and nothing else, so under ROS Noetic the binary initialises, subscribes to nothing, and exits immediately even with data flowing. Use the `bag` binary and a recorded rosbag, which is the path `subscribe.launch`’s own hard-wired `rosbag play` node implies. And do not quote pose accuracy from such a run unless the camera–IMU extrinsic has been calibrated and the IMU units corrected (Chapter 7) — with a placeholder extrinsic and accelerations in *g*, the filter diverges grossly while the timing figures stay valid.

9.1.4 MINS on the ground station (lab PC)

MINS’s deployment target is the lab PC, not the Pi. The relocalisation work this manual builds towards fuses MINS with Carolus on a ground station that receives the robot’s data over the network, not on board the robot itself — the Pi build above remains useful (it is inside MINS’s own tested OS/ROS combination, Section 9.1) but is a secondary path, not the one this section documents.

MINS is built in its own catkin workspace on the lab PC (`mins_sandbox/`), kept entirely separate from the workspace this manual otherwise describes — nothing under `carolus_ws` depends on it, and nothing here should be built alongside the main pipeline. This is also separate from the T5 MINS (simulation, Pi) launcher button (Section 11.1), which runs MINS’s own bundled simulation demo on the Pi’s `~/mins_sandbox_ws` and is unaffected by anything in this section — both are real, both are separate MINS installs for separate purposes, and neither depends on the other.

Verified 2026-08-11: the workspace builds cleanly, and the camera-image boundary has been tested against real data, not only compiled. Three fixes were needed on this exact machine (Ubuntu 22.04.5) that a 20.04/Noetic build does not need:

1. **C++ standard.** `ov_core` and `ov_eval` request C++14; on this machine, `ros/console.h` pulls in a `log4cxx` header that requires `std::shared_mutex` (C++17). Raise `CMAKE_CXX_STANDARD` to 17 in both packages' `CMakeLists.txt`.
2. **A consistent OpenCV build throughout the workspace.** This machine exposes a single OpenCV development package, version 4.5.4. The ROS-distributed `cv_bridge` package, however, is linked against a separate, older OpenCV 4.2 set of libraries that Ubuntu 22.04 also carries for compatibility. Building MINS's own code against 4.5.4 while linking the distributed `cv_bridge` mixes two different OpenCV builds in one process — this is unsafe and has been observed to crash (not on every code path, but reproducibly on the one that matters: converting a camera image and running feature tracking on it). **Fix: build `cv_bridge` and `image_geometry` from source (ros-perception/vision_opencv, noetic branch) inside `mins_sandbox/catkin_ws/src/`, so they resolve to the same 4.5.4 as the rest of the workspace:**

```
cd mins_sandbox/catkin_ws/src
git clone --depth 1 -b noetic \
  https://github.com/ros-perception/vision_opencv.git
rm -rf vision_opencv/opencv_tests # not needed
catkin build cv_bridge image_geometry
```

One further fix is needed in this freshly-cloned `cv_bridge`: its own `src/CMakeLists.txt` calls `boost::regex` without linking `libboost_regex` (an upstream gap, not specific to this project). Add `boost_regex` to that target's `target_link_libraries` call before building.

3. **Build the rest of the workspace on top of the fixed `cv_bridge`**, sourcing its `devel/setup.bash` first so later packages resolve against it rather than the system one:

```
source /opt/ros/noetic/setup.bash
cd mins_sandbox/catkin_ws
source devel/setup.bash
catkin build
```

4. **An unambiguous `log4cxx` link.** This machine carries two major versions of `liblog4cxx` side by side — `liblog4cxx10v5` (0.10.x, what ROS Noetic's own prebuilt `librosconsole_log4cxx.so` expects) and the system's native `liblog4cxx-dev/liblog4cxx12` (0.12.x). The unversioned `/usr/lib/x86_64-linux-gnu/liblog4cxx.so` symlink resolves to the 0.12 build, and `mins`'s own `cmake/ROS1.cmake` re-links every entry of `catkin_LIBRARIES` by default — including that symlink — giving `mins_lib.so` a direct dependency on 0.12 alongside ROS's own 0.10, both loaded in the same process. Two incompatible major versions of the same C++ library coexisting this way segfaults during static initialisation the first time anything touches `ros::console`, i.e. immediately on node start. **Fix: in `mins/mins/cmake/ROS1.cmake`**, extend the existing “drop missing `catkin_LIBRARIES` entries” filter with a name match that also drops any entry matching `liblog4cxx\so$` (the ambiguous symlink specifically, not the versioned files) before the filtered list is appended to `thirdparty_libraries`. MINS's own source calls no `log4cxx` symbol directly, so the correct 0.10 build still reaches the process — supplied transitively, exactly once, through `roscpp`.

What “verified” means here, precisely. All eight packages build without error. The `cv_bridge` image-conversion boundary specifically — converting a `sensor_msgs/Image` and running the same corner-detection and optical-flow calls MINS's own tracker makes on it — was re-run three times against real recorded camera frames (twice directly, once under a debugger) after the `cv_bridge` rebuild, with no crash. With the `log4cxx` fix above also in place, MINS's own bundled simulation (`roslaunch mins`

`simulation.launch`, no robot or bag needed) runs its estimator end to end and writes a genuine trajectory file plus IMU comparison files to `outputs/tmp/` — the first time this build has produced estimator output on this machine, not merely compiled. **One caveat carried over from that same run:** the `simulation` process still segfaults during its own shutdown, after the estimator has already finished and written its output — a separate, still-open ABI conflict (a ROS-distributed `image_transport` plugin, `compressed_image_transport`, links an older OpenCV/GDAL/PROJ chain than the rest of this build and is loaded via `pluginlib` at run time, invisibly to `ldd`). `roslaunch` will report the node as having died; the output files under `outputs/tmp/` are unaffected and worth checking directly rather than trusting the exit status. **This is still not the same as a validated pose estimate on this robot.** The rectangular-motion experiment that exercises the full estimator against a measured ground truth, on this robot’s own sensor data, has not been run yet; nothing in this section should be read as MINS’s accuracy being confirmed end to end outside its own bundled simulation dataset.

9.2 Getting this project’s code onto the machine that will build it

Clone the project’s repository onto whichever machine will build Carolus — **normally the Raspberry Pi** (Section 9.1), over SSH from the lab PC (Chapter 5). The clone’s own top level holds this manual (`overleaf/`) and the operator tooling (`shortcuts/`, not covered in this manual) alongside a `carolus_ws/` directory, which is the actual catkin workspace — a standard layout, with the ROS packages under `carolus_ws/src/`: `carolus_ws/src/carolus_node/` (the launch files and configuration), `carolus_ws/src/libuvgs_astrobeek/` (Carolus’s own C++ source, already patched for this robot — it builds a ROS-free algorithm library plus the ROS node that wraps it, see Section 9.4), `carolus_ws/src/ff_msgs/` (a message-definition dependency), `carolus_ws/src/robomaster_cam/` (the Pi-side bridge, Chapter 7), plus `carolus_ws/cmake_shims/` (Ubuntu 22.04 build fixes, see below):

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git
# or, over HTTPS -- the repository is public, no credentials needed:
# git clone https://github.com/Arracheur2Bonnet/robotmaster.git
```

If you already have a clone from an earlier session, update it instead of cloning again — fixes to this manual’s own corrected code (such as Section 9.3 below) live in the repository’s git history, not only in this document, and an old clone will keep failing in exactly the way a fix here describes until it is pulled:

```
cd robotmaster # or whatever you renamed the clone to
git pull
```

This creates a `robotmaster/` directory; `cd` into it before running any command below that assumes `carolus_ws/` is the current directory’s subdirectory, or rename it to whatever you prefer.

Repository web address: <https://github.com/Arracheur2Bonnet/robotmaster>

The repository is public. Both clone commands above work as-is, from any machine, with no access request and no credentials needed to read it. The SSH setup below is still worth doing even so — it avoids retyping credentials on every push, and it is required the moment you are given write access.

9.2.1 Setting up an SSH key

Cloning over SSH (the first command above) needs an SSH key registered against *your own* GitHub account. This is a one-time setup per machine, and it is what replaces the practice — found in some older inherited documents — of pasting a shared personal access token directly into a clone command. A shared token grants whoever holds it the full access rights of the account that created it, cannot be

attributed to an individual, and stays valid in every copy of the document that carries it.

1. Generate a key on the machine that will clone the repository (the Raspberry Pi and the lab PC each need their own):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept the default path (~/.ssh/id_ed25519).
# A passphrase is optional but recommended.
```

2. Display the **public** half of the key — this is the only part that ever leaves the machine. Never share the file without the `.pub` extension:

```
cat ~/.ssh/id_ed25519.pub
```

3. On GitHub: *profile photo* → *Settings* → *SSH and GPG keys* → *New SSH key*. Give it a title identifying the machine (e.g. “RoboMaster Pi”), paste the full line printed above, and save.
4. Verify the key works before attempting the clone:

```
ssh -T git@github.com
```

A successful setup answers with `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.` — that message is the expected result, not an error.

If you prefer HTTPS to SSH, use the second clone command instead and authenticate with your GitHub username and a personal access token *that you generate for your own account* — created under *Settings* → *Developer settings* → *Personal access tokens*, scoped to repository read access, and never shared with anyone else or written into a document.

9.3 Installing the workspace’s remaining dependencies

`ros-noetic-ros-base` (Section 5.7) does not include everything this project’s own packages need, and neither the perception stack nor the solver libraries come with it. Between them, `robomaster_cam`, `carolus_node` and `libuvgs_astrobees` require `image_transport`, `cv_bridge` and `image_geometry` (which belong to `ros-noetic-perception`), plus **Ceres** and **Eigen3**, which are ordinary system libraries rather than ROS packages at all.

Building without them fails at CMake’s configure step, naming only the *first* thing it cannot find — so fixing one error simply reveals the next. Two real examples, both reported from a fresh Raspberry Pi: `Could not find image_transport (missing: image_transport_DIR)`, then, once that was resolved, `Could not find a package configuration file provided by "Ceres"`. Note also that because `catkin_make` configures the *entire* `src/` tree in one pass, either error appears even when building an unrelated package such as `ff_msgs`, which needs none of them.

Fix: let `rosdep` resolve every package’s real dependencies in one go, rather than installing them one CMake error at a time. Run this once from the workspace root (the clone from the section above, updated with `git pull` if it already existed), before either build path below:

```
cd ~/carolus_ws
rosdep install --from-paths src --ignore-src -r -y
```

On a blank Ubuntu 20.04 machine that installs `ros-noetic-cmake-modules`, `libeigen3-dev`, `libyaml-cpp-dev`, `python3-catkin-pkg`, `libopencv-dev` and `libceres-dev` — verified by building this workspace from

nothing in a clean container (`shortcuts/cleanroom_build.sh`). If `rosdep` is unavailable or misbehaves, the equivalent direct install is:

```
sudo apt install ros-noetic-image-transport ros-noetic-cv-bridge \
  ros-noetic-image-geometry libceres-dev libeigen3-dev
```

If you hit a missing dependency this section does not mention, that is a defect in this manual — please report it rather than working around it. Send the exact CMake error line. Two such gaps have been found and closed this way, and the fix belongs in the project rather than in your local setup: once the dependency is declared properly, the `rosdep install` above installs it for everyone, instead of each reader rediscovering it one error at a time.

9.4 What the package builds: two targets, not one

`libuvgs_astrobees` builds **two** things, and the split matters if you are editing the C++ or porting it elsewhere:

Target	Sources	Links against
<code>carolus_core</code> (static lib)	<code>beacon_detector.cpp</code> , <code>ceresP4P.cpp</code> , <code>pose_est.cpp</code> , <code>pose_filter.cpp</code>	OpenCV, Eigen, Ceres — no ROS at all
<code>carolus_astrobees</code> (node)	<code>carolus_astrobees.cpp</code>	<code>carolus_core</code> + catkin/ROS

`carolus_core` holds the algorithm: blob preprocessing, contour extraction and selection (`BeaconDetector`), the target sort, the Ceres P4P solve (`CobrasFumantes`), and the FIFO outlier filter (`PoseFilter`). It is verifiably ROS-free — the compiled library’s `NEEDED` list is empty and `nm -uC` reports zero unresolved ROS symbols.

`carolus_astrobees.cpp` is the ROS wrapper around it: subscriptions, publishers, parameter loading, the image queue, and the conversions between ROS types and the core’s plain C++/Eigen/OpenCV ones.

When editing the core, keep it that way. It links no ROS libraries, so `ROS_INFO`/`ROS_WARN`/`ROS_ERROR` are unavailable inside it by design — `BeaconDetector` and `PoseFilter` both take an injected logger callback in their constructor instead, which the node wires to the ROS macros. Adding a ROS include to a core file breaks the property the split exists for, and the build will not warn you: it will simply succeed and quietly pull ROS back in.

The core takes timestamps as `int64_t` nanoseconds, not `double` seconds. That is not stylistic: an IEEE-754 double at a 2026-era epoch value has a spacing of roughly 238 ns between representable values, so `t + 1 ns == t` evaluates true and a ROS timestamp cannot round-trip through it. The node converts once, at its own boundary (`timestamp.toNSec()`).

9.5 Building on Ubuntu 20.04 (the Raspberry Pi — normal case)

This is the normal path: the Raspberry Pi runs Ubuntu 20.04, ROS Noetic’s officially supported target, so the build itself needs no compiler/ABI workarounds the way Section 9.6 does. (A lab PC also on 20.04 builds identically.) **It does still need the `rosdep install` step from Section 9.3 above run**

first, if not already done — skipping it fails here with `Could not find image_transport (missing: image_transport_DIR)` or a similarly-named missing package.

```
cd ~/carolus_ws
rosdep install --from-paths src --ignore-src -r -y # if not already run
catkin_make --pkg ff_msgs # build this dependency first, avoids ordering
issues
catkin_make
source devel/setup.bash
```

That is the whole build. Skip the 22.04 section entirely — it exists only for a lab PC on an unsupported OS — and go to Section 9.7.

On a Pi, expect the compile to take several minutes. If memory is tight on a 4GB model, pass `-j2` to limit parallel jobs.

9.6 Building on Ubuntu 22.04 (lab-PC fallback only)

Skip this section unless you are building on a lab PC running Ubuntu 22.04. None of it applies to the Raspberry Pi.

ROS Noetic officially targets Ubuntu 20.04 only. On 22.04, four system-level fixes are needed before `catkin_make` succeeds, and a fifth at launch time. They are recorded because this project’s lab PC was upgraded to 22.04 unintentionally and the workarounds took real effort to find — not because this is a recommended configuration.

Package	22.04 (this machine)	20.04 (Pi)	Needs
Ceres	2.0.0+dfsg1-5	1.14.0-4ubuntu1.1	no fix, both work
Boost	1.74.0-14ubuntu3	1.71 (native)	real 1.71 .so files (Fix 3)
ICU	66 + 70	67 (native)	real 1.67 .so files (Fix 4)

The same `liblog4cxx 10-vs-12` collision behind Fix 5 recurs elsewhere on this machine whenever a C++ node built here loads ROS’s prebuilt console backend — MINS hit it independently under a different build system (Chapter 12, Fix 4), fixed at the CMake level there rather than with `LD_PRELOAD`. Same root cause, two valid fixes depending on whether the build system is yours to patch.

9.6.1 Fix 1: glibc symlinks

`librt`, `libpthread`, and `libdl` were merged into `libc.so.6` starting with glibc 2.34 (shipped in 22.04); older binaries and build systems still look for the separate files.

```
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
librt.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
libpthread.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
libdl.so
```

9.6.2 Fix 2: libpython3.8 symlink

Referenced by `rospack` (an official ROS Noetic package):

```
sudo ln -sf /usr/lib/x86_64-linux-gnu/libpython3.8.so.1 /usr/lib/x86_64-linux-  
gnu/libpython3.8.so
```

9.6.3 Fix 3: Boost 1.71 (real libraries, not symlinks)

ROS Noetic's official binaries need Boost 1.71; 22.04 ships 1.74, which is ABI-incompatible for the libraries ROS actually links (regex, filesystem). Symlinking will not work here — the real 1.71 .so files are needed:

```
# Download from http://old-releases.ubuntu.com/ubuntu/pool/{main,universe}/b/  
boost1.71/  
# the following packages: libboost-{chrono,date-time,filesystem,program-  
options,  
# regex,system,thread}1.71.0  
  
dpkg-deb -x <package>.deb <extract_dir>  
# copy each extracted .so.1.71.0 file into /usr/lib/x86_64-linux-gnu/  
sudo ldconfig
```

Pitfall: if a symlink already exists at the destination path (pointing at the 1.74 library), a plain `cp` writes *through* the symlink and corrupts the real system 1.74 file. Always remove the symlink first:

```
sudo rm -f /usr/lib/x86_64-linux-gnu/libboost_<name>.so.1.74.0    # if it is a  
symlink  
# then copy the real 1.71 file to that path
```

If this happens anyway, recovery is a straightforward package reinstall:

```
sudo apt install --reinstall libboost-{chrono,date-time,filesystem,program-  
options,regex,system,thread}1.74.0
```

9.6.4 Fix 4: ICU 67

A dependency of Boost 1.71's regex component; 22.04 ships 66/70, not 67.

```
# Download libicu67 from old-releases.ubuntu.com  
# copy libicuuc.so.67*, libicui18n.so.67*, libicudata.so.67* into /usr/lib/  
x86_64-linux-gnu/  
sudo ldconfig
```

9.6.5 Building, after the four fixes

```
cd ~/carolus_ws  
rm -rf build/ devel/  
catkin_make --pkg ff_msgs  
catkin_make  
source devel/setup.bash
```


9.6.6 Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)

A binary compiled on 22.04 links against `liblog4cxx.so.12` and `libtinyxml2.so.9`; ROS Noetic's own libraries (`libroscconsole.so`, `libimage_transport.so`) expect `.so.10/.so.6`. Both versions load at runtime and the 22.04 version wins global symbol resolution, which segfaults inside `ros::console::initialize()`. The fix, scoped to the affected node in its launch file (already applied in the launch file below, and in `ekf_lite.launch`):

```
1 <env name="LD_PRELOAD"
2   value="/usr/lib/x86_64-linux-gnu/liblog4cxx.so.10:/usr/lib/x86_64-linux-
   gnu/libtinyxml2.so.6" />
```

Apply the same pattern to any other C++ node recompiled under 22.04 that segfaults the same way on startup.

9.6.7 If an additional ROS package needs to be built from source

Packages unavailable via `apt` on 22.04 (e.g. `robot_localization`, for planned EKF sensor-fusion work) can fail with CMake errors about a missing `<pkg>Config.cmake` or a nonexistent `orocos_kdl` include path — seven packages (`eigen_conversions`, `tf2_geometry_msgs`, `kdl_parser`, `kdl_conversions`, `robot_state_publisher`, `tf2_kdl`, `tf_conversions`) are broken this way by the 20.04→22.04 packaging difference. Without root access to fix the real files under `/opt/ros/noetic/`, the working solution is a set of locally corrected copies in `carolus_ws/cmake_shims/<pkg>_fix/`, given priority via `list(PREPEND CMAKE_PREFIX_PATH ...)` in the `CMakeLists.txt` of whichever package is being built.

Stale-cache pitfall: if a shim is created only after a first failed build attempt, `<pkg>_DIR` may stay cached with the old broken path. Clear just that variable rather than wiping the whole build:

```
cmake -U <pkg>_DIR
```

9.7 Launching Carolus

Carolus's parameters are split in two, so the node can run on other robots without editing any code:

- `carolus_ws/src/carolus_node/config/robomaster_s1.yaml` — everything specific to *this* robot and *this* beacon: camera intrinsics, distortion, beacon geometry, blob-detection tuning.
- `carolus_ws/src/carolus_node/launch/carolus.launch` — the generic launch file. It contains no hardware values at all; it loads a profile and wires up topics.

`testcarolus.launch` is the RoboMaster S1 entry point: it selects the S1 profile, adds an S1-specific static transform, and delegates to `carolus.launch`.

9.7.1 The hardware profile

The values below are this robot's. **Every one of them must be re-measured for different hardware** — see Chapter 8 for the camera intrinsics, and the note on `known_points` at the end of this chapter for the beacon.

```
1 # --- Camera intrinsics (MEASURE THESE FOR YOUR OWN CAMERA) ---
2 fx: 546.1957
3 fy: 547.0838
```

```

4 cx: 575.6041
5 cy: 372.1876
6 distortion: [-0.1479, 0.1452, -0.0023, 0.0025] # k1 k2 p1 p2, a real YAML
7 # list. A quoted string
8 # here silently loads a
9 # hardcoded placeholder
10 # instead, with no warning.
11
12 # --- Beacon geometry (MEASURE THIS FOR YOUR OWN BEACON) ---
13 # Four LED positions in metres, Carolus's left-handed frame. One point is
14 # deliberately off the plane of the other three -- P4P needs that.
15 known_points: "0.0825 0.0 0.0 -0.0825 0.0 0.0 0.0 0.072 0.0 0.0 0.0 0.0555"
16
17 # --- Blob detection ---
18 min_circularity: 0.6
19 saturation_threshold: 80
20 min_area: 8.0 # see the note below before changing this for range
21 max_area: 1800.0
22 max_distance_lim: 1000.0
23 lb_hue: 90.0 # OpenCV 0-180 scale, NOT 0-360. Blue LEDs here.
24 ub_hue: 140.0
25
26 # --- Preprocessing ---
27 kernel_size_gaussian: 3
28 kernel_size_morph: 3
29 image_threshold: 190
30
31 # --- Outlier rejection (moving window over successive poses) ---
32 filter_size: 7
33 translation_threshold: 0.5
34 rotation_threshold: 0.3
35 fifo_on: true
36
37 # --- Execution ---
38 num_threads: 1
39 queue_size: 10
40 benchtest: true
41
42 # --- Camera selection (Astrobee-era flags, kept for compatibility) ---
43 nav_cam: true
44 dock_cam: false
45 fisheye: false
46 fov: true
47 mono: false
48 bot_name: "wannabee" # inert legacy label, no functional effect

```

9.7.2 Launching

On the Raspberry Pi (the normal case). The `ubuntu2204_preload:=false` argument is required: the LD_PRELOAD workaround it disables exists only for the 22.04 library mismatch (Fix 5), and it hardcodes `x86_64` paths, so leaving it on would be not merely unnecessary but wrong on `aarch64`.

```

# On the Pi, with the camera bridge already running (Chapter 6), in a
# SEPARATE shell -- ROS_MASTER_URI/ROS_IP are per-shell, not inherited:
source /opt/ros/noetic/setup.bash

```

```
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.103
source ~/carolus_ws/devel/setup.bash
roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false
```

Carolus and the camera bridge then both run on the Pi, and no image crosses the network — which is what makes the robot usable away from the lab PC, and what gives the camera input rate measured in Section 9.1.2.

On a lab PC running Ubuntu 22.04 (development only, per Section 9.1), the master is still on the Pi — `ROS_MASTER_URI` points there, `ROS_IP` is the PC's own address (Chapter 6), and the preload argument is omitted so it stays enabled:

```
export ROS_MASTER_URI=http://192.168.0.103:11311
export ROS_IP=192.168.0.100
roslaunch carolus_node testcarolus.launch
```

On a different robot entirely, copy the profile, substitute your own measured values, and point Carolus at your own camera topic — and set `ROS_MASTER_URI/ROS_IP` for *that* robot's own network, not this project's addresses above. No source file and no launch file needs editing:

```
roslaunch carolus_node carolus.launch \
  config:=$(rospack find carolus_node)/config/my_robot.yaml \
  camera_topic:=/my/camera/image_raw \
  ubuntu2204_preload:=false
```

The `robomaster_cam` package (Chapter 7) is the DJI SDK bridge for this specific robot and is **not** needed on other hardware — supply the camera stream however your own platform already does.

9.7.3 The pose timestamp: `stamp_from_acquisition`

`/pose`'s header is stamped with the ACQUISITION time of the image it was computed from, not the time it was published — **leave this at its default of true**. The alternative (`stamp_from_acquisition:=false`) reproduces the old behaviour, where every pose is stamped `ros::Time::now()` at the moment Carolus finishes computing it. That difference is not cosmetic: a fusion filter reads `header.stamp` to place the measurement in time, so the old behaviour hands it a systematic offset it has no way to detect, and it also makes the pose's real latency unmeasurable from ROS alone — the gap between capture and publication silently disappears into "now".

```
roslaunch carolus_node testcarolus.launch stamp_from_acquisition:=false
```

The only legitimate reason to set it **false** is if the image timestamps come from a machine whose clock is not synchronised with whatever consumes `/pose` — and even then, fix the clock sync rather than the stamp. **Check this before trusting it:** the Pi's system clock is not guaranteed to be NTP-synchronised (observed drifted by several days on this project, 2026-08-10), and a drifted clock corrupts this exact mechanism silently, the same way the old publish-time stamping did.

9.8 Verifying it is running

```
rostopic info /carolus_astrobeerex
rostopic hz /pose
rostopic hz /postprocessed/image
```

With a beacon visible to the camera, `/pose` publishes a `geometry_msgs/PoseStamped`. Visualize blob detection in RViz by adding an **Image** display on `/postprocessed/image`: correctly detected blobs appear as small circles over the live feed.

Check the latency, not just the rate. A high `/pose` rate says nothing about whether `stamp_from_acquisition` landed correctly (previous section). In the camera bridge’s own log (`rm_cam_beacon.py`, Chapter 7), every 50 poses prints `[LATENCY] /pose NN ms (mean over 50 poses)` — a plausible value confirms acquisition-time stamping and synchronised clocks. A value outside `-1-5000 ms` instead logs `[LATENCY] implausible (NN ms)` -- `/pose` is probably stamped at publication time, not acquisition, naming the same two causes: an old binary running without the flag, or the two machines’ clocks disagreeing.

Checkpoint — the core pipeline is now complete. With a powered beacon in the camera’s view, `rostopic hz /pose` reports roughly 13Hz on the Pi. No beacon in view means no pose at all — that is correct behaviour, not a fault. Once this passes, go to Chapter 11 to actually drive the robot.

9.9 Beacon geometry and blob-filtering parameters

The parameters most likely to need tuning for a new beacon or lighting condition:

Parameter	What it does
<code>num_threads</code>	How many CPU cores the node may use. Raising it can cut latency; on a 4-core Pi, 2 is already half the machine.
<code>queue_size</code>	How many incoming images ROS buffers before dropping the oldest. If detection is slower than the camera publishes, a larger queue stops frames being dropped — at the cost of processing older ones.
<code>min_circularity</code>	How round a blob must be, scored 0–1. Closer to 1 is stricter.
<code>saturation_threshold</code>	Minimum HSV saturation (0–255) a pixel must have to be considered at all, applied before hue filtering.
<code>min_area, max_area</code>	Valid blob size range in pixels. Anything outside is rejected. This is the parameter that sets the maximum working distance , because a blob’s area falls off as the inverse square of range: the LEDs measure 85–131 px ² at 0.78 m on this robot (Measured 2026-08-14), which scales to roughly 0.8–1.3 px ² at 8 m. But it cannot simply be lowered. Set to 0.4 for that range and then measured at 1 m, it admits a marginal 2.5 px ² contour into the four-blob selection and the P4P solution becomes ambiguous — 65% of samples landed on the solver’s alternate near-planar solution, against 36% at min_area=8.0 (Measured 2026-08-14, same beacon angle, only this parameter changed). It is therefore back at 8.0. Reaching 8 m needs a compromise value or a distance-aware rule, not one global threshold: the two ends of the working range pull in opposite directions, which the Limo integration’s own document also warns about.
<code>max_distance_lim</code>	Maximum pixel distance beyond which candidate blobs are discarded.
<code>lb_hue, ub_hue</code>	Accepted hue range in HSV, in OpenCV’s 0–180 scale , not 0–360. Any blob whose colour falls outside is rejected.
<code>kernel_size_gaussian</code>	Gaussian blur kernel size — reduces sensor noise and smooths the image before thresholding.
<code>kernel_size_morph</code>	Morphological kernel size — cleans the binary mask, removing speckle and filling small holes inside a blob.
<code>image_threshold</code>	Brightness cut (0–255) isolating bright emissive regions before colour filtering. Beacon LEDs are bright, so this removes most of the scene cheaply.

9.9.1 The pose filter

A second, separate group of parameters governs what happens *after* a pose is solved. Every solved pose goes through a FIFO outlier filter (**PoseFilter**, Section 9.4) before publication: a new pose too far from the last accepted one is rejected outright, and what gets published on `/pose` is the average over a sliding window, not the raw single-frame solve.

Parameter	Value	What it does
<code>fifo_on</code>	<code>true</code>	Master switch. With it off, each solved pose is published raw.
<code>filter_size</code>	7	Window length in poses. Translation is averaged arithmetically, rotation by incremental quaternion SLERP.
<code>translation_threshold</code>	0.5 m	A new pose further than this from the last accepted one is rejected as an outlier.
<code>rotation_threshold</code>	0.3 rad	Same gate on angular distance.
<code>reject_limit</code>	5	After this many <i>consecutive</i> rejections the window is dropped entirely and rebuilt — the escape hatch for when the target has genuinely jumped rather than glitched.
<code>max_time_fifo</code>	5.0 s	If this long passes with no accepted pose, the window is discarded rather than averaged across the gap.

The published pose is an average, so it lags. With `filter_size` at 7 and the pipeline near 13 Hz, `/pose` trails the real target by roughly half a window. That is the intended trade (smoothness against latency), but it means `/pose` is not the right signal for judging raw detection quality — read the node’s own per-frame log lines for that, or set `fifo_on` to `false` temporarily.

The stamp on the published pose is the *acquisition* time of the most recent image that contributed to the average, not the publication time.

Provenance: the per-parameter descriptions above are **inherited**, from the Leo Rover integration’s own write-up of the same Carolus node. They describe what each parameter does, which is platform-independent; the *values* in that document are for a different camera and beacon and are not reproduced here. This project’s own values are listed in Section 9.7.

The hue scale is worth stating explicitly, because getting it wrong silently kills all detection. OpenCV stores hue on 0–180 (half the usual 0–360 degree circle, so it fits in one byte), and the node compares the blob’s mean hue against `lb_hue/ub_hue` on that same 0–180 scale. Rough reference points: red $\approx$ 0–10 and 170–180, green $\approx$ 50–80, blue $\approx$ 100–140. This project’s beacon uses blue LEDs, hence the 90–140 window in the launch file. If you enter a value from the 0–360 convention (e.g. 240 for blue), every blob is rejected and the node runs normally while never publishing a pose.

This beacon has a physical LED intensity control. Turning it changes every LED’s brightness at once, which shifts the blob’s measured saturation and can push it outside `saturation_threshold`, `min_area`/`max_area`, or the hue window above — **the whole table on this page is only valid for the intensity the beacon was set to when these values were tuned.** If the knob has been touched since, or you are not sure, re-run the detection tuning (adjust the parameters above while watching `[BEACON] status=DETECTED` in the launcher log) before trusting any pose Carolus reports. This does not affect `known_points` below, which is a geometric measurement, not a brightness one — but a beacon that has been physically disassembled or rebuilt needs that re-measured too.

Too bright looks exactly like “no beacon” in the log. Read this before diagnosing a detection failure. Measured 2026-08-17: with the intensity set high, the four LEDs bloom and merge into a single irregular blob covering **88 628 saturated pixels** — **9.6% of the whole frame**. That blob is rejected by `max_area` for size and by `min_circularity` for shape, so Carolus reports Not enough blobs with required circularity and then 0 contours found. **Both messages point an operator towards a missing or unlit beacon, when the cause is the exact opposite.** If detection has stopped and the beacon is plainly lit, check the brightness before anything else.

`shortcuts/beacon_brightness_live.py` exists for this: it prints the saturated-pixel count, the hue, and the detection rate on one refreshing line, so the physical knob — which has no readout of its own — can be turned against live feedback. **Tune to maximise the detection rate, not to hit a pixel count:** on the same day a setting that looked plausible by brightness alone was delivering 1.5 Hz where 5.4 Hz was achievable. A known-good signature is recorded in `carolus_node/config/beacon_reference.yaml`.

The LEDs are PWM-driven, which shapes the trade-off. Across 333 consecutive frames the saturated-pixel count ranged from **0 to 37 182**, and a frame reading exactly zero is not noise — it is the camera sampling the off phase of the duty cycle. So turning the intensity down does not merely shrink the blobs, it lowers the detection *rate*, because more frames land mid-off and fail outright. There is a floor below which detection degrades regardless of how well-sized the blobs look.

9.9.2 Operating rules for the beacon

Detection tuning is only reproducible if the beacon is operated the same way each time. The rules below cost nothing to follow and invalidate a whole session when broken.

Everything here is inherited from the two prior Carolus integrations (Leo Rover and LIMO), which ran the same node on different robots, cameras and beacons. Their *practices* transfer; their *numbers* do not, and are marked where quoted. Nothing in this subsection has been re-measured on the RoboMaster.

- **Always use the same beacon at the same LED intensity.** Changing either means the detection parameters must be re-tuned from scratch — the prior integration states this as a hard warning, not a caution: with a different intensity “Carolus will no longer work”.
- **Keep the beacon charged. Inherited figure**, from the LIMO integration and not verified here: do not run below **10.5 V**. Readings change as the battery drains, so a session that starts calibrated can drift out of calibration purely through discharge.
- **Brightness is a trade-off, in two directions at once.** Too intense costs battery life; too dim and detection fails. The prior work also gives a tuning order worth reproducing: **set the LED intensity for good readings at the *shortest* working distance, then set the blob area for good readings at the *longest*.** The two ends of the range pull against each other, so tuning them in that order avoids chasing one at the expense of the other.
- **If the beacon is not detected at all**, the first parameter to try is `max_area` — reduce it, then adjust the rest around it.
- **Do not tune in the dark.** A darkened room makes detection easier and produces parameters that then fail under the lighting the robot will actually work in. Tune in the light level you intend to operate in.

The calibration checkerboard is printed on the back of the beacon itself. There is no separate target to print for the camera calibration in Chapter 8 — turn the beacon around. This is a property of this project’s specific beacon, confirmed 2026-08-11.

`known_points` is the beacon’s four LED positions in metres, in Carolus’s own left-handed frame, with one point deliberately off the plane of the other three (needed for P4P to resolve pose unambiguously). **These are a physical measurement of this project’s specific beacon, not a universal constant** — the same caveat as the camera intrinsics in Chapter 8. A beacon built to different dimensions needs these four points re-measured and substituted; reusing this project’s values against a differently-sized beacon will make the P4P solve fail silently or produce a plausible-looking but wrong pose.

The *sign* of the out-of-plane coordinate matters more than its magnitude, and getting it wrong is nearly undetectable. Recording the fourth LED on the wrong side of the plane of the other three — writing `+0.0555` where the hardware has it at `-0.0555` — describes a beacon that is the mirror image of the real one. The solver still converges, the residual still looks healthy, and the reported pose is still a valid rotation. What changes is its handedness: rotations about two of the three axes come back with **inverted sign**, while the third reads correctly.

Demonstrated numerically rather than assumed (2026-08-11): projecting the real four-LED geometry and re-solving with that one coordinate’s sign flipped inverts the recovered rotation about X and Y while leaving Z correct. By contrast, corrupting the *camera intrinsics* severely — a 25% focal-length error, a 60-pixel principal-point shift, and distortion ignored entirely — shifts the recovered angles by only a few percent and **never flips a sign**.

Practical consequence: if orientation readings look mirrored, suspect this value’s sign before suspecting the camera calibration. Measure which side of the plane the fourth LED sits on and confirm it against the sign written here.

Chapter 10

Converting a Carolus Pose into the ROS Frame

Runs on: the machine running the TF broadcaster — normally the **Raspberry Pi**, alongside Carolus itself.

Carolus reports where the beacon is *in its own camera frame*. Every ROS consumer downstream — TF2, RViz, any sensor-fusion filter — expects REP-103 conventions instead. This chapter derives that conversion rather than just stating the constant, because the constant alone is impossible to check and easy to get subtly, silently wrong.

10.1 Why this is harder than it looks

Two independent earlier integrations of Carolus, on a Leo Rover and on a Limo, each arrived at their conversion constants by trial and error against real hardware. Both worked on their own platform. Neither derivation was written down, so neither transfers, and there is no way to tell from the constants alone whether they encode a genuine convention difference or a compensating error.

The failure mode this invites is specific and nasty: an orientation conversion that is wrong can still produce a perfectly valid unit quaternion. Nothing raises an error, RViz draws a plausible set of axes, and the mistake only surfaces as motion in the wrong direction much further downstream.

10.2 Step 1 — what Carolus actually publishes

Carolus’s P4P solver minimises reprojection error in the standard computer-vision form: for each known beacon LED at position $\mathbf{p}_b$ in the *beacon’s* own frame, it predicts a pixel by

$$\mathbf{p}_c = R_{\text{solved}} \mathbf{p}_b + \mathbf{t}, \quad u = f_x \frac{p_{c,x}}{p_{c,z}} + c_x, \quad v = f_y \frac{p_{c,y}}{p_{c,z}} + c_y.$$

R_{solved} therefore maps *beacon-frame* coordinates into *camera-frame* coordinates: it is $R_{\text{beacon} \rightarrow \text{camera}}$. This is the same convention OpenCV’s `solvePnP` documents for its own output, and the two are interchangeable as a sanity check.

But that is not what reaches the ROS topic. Before publishing, Carolus transposes it. What appears in the `/pose` message’s `orientation` field is

$$R_{\text{published}} = R_{\text{solved}}^{\top} = R_{\text{camera} \rightarrow \text{beacon}}.$$

This transpose is the single most commonly missed step. A quaternion built from $R_{\text{published}}$ describes the *opposite* rotation to the one the solver computed. Both are valid quaternions; only one is the one you want.

10.3 Step 2 — what TF2 expects

The broadcaster publishes the edge `camera_link` $\rightarrow$ `beacon_observed`. By TF2’s convention, the rotation on a parent $\rightarrow$ child edge must take a vector expressed in the *child* frame and give it in the *parent* frame. Here that is `beacon` $\rightarrow$ `camera`:

$$R_{\text{needed}} = R_{\text{beacon} \rightarrow \text{camera}} = R_{\text{published}}^{-1}.$$

So the published quaternion must be **inverted** before anything else is done to it. Combined with Step 1, the two transposes do not cancel: the solver transposes once on the way out, and the TF convention requires inverting once on the way in.

10.4 Step 3 — the change of basis, and why it is proper

10.4.1 Carolus’s camera frame, stated definitively

Carolus’s axes are ***X* right, *Y* down, *Z* forward** (out of the lens). This is not a convention chosen for this document: it follows from the projection equations of Step 1, where $u = f_x p_{c,x}/p_{c,z} + c_x$ grows with *X* toward increasing pixel columns (rightward) and $v = f_y p_{c,y}/p_{c,z} + c_y$ grows with *Y* toward increasing pixel rows (downward), with $p_{c,z}$ the depth in front of the camera.

It is also, exactly, the convention ROS itself specifies for camera frames. REP-103, *Standard Units of Measure and Coordinate Conventions*, §“Suffix Frames”:

“In the case of cameras, there is often a second frame defined with a ‘_optical’ suffix. This uses a slightly different convention: z forward, x right, y down.”

So a Carolus pose is already in a standard ROS optical frame, and the correct TF frame name for it ends in `_optical`. Everything below converts from that to REP-103’s *body* convention (*X* forward, *Y* left, *Z* up), which is what the rest of the ROS stack expects.

10.4.2 Carolus is NOT the Intel D455 convention, and NOT the SVGS one

Two other diagrams circulate for this hardware, and **neither matches Carolus**. Copying rotations between them without converting silently inverts axes.

Frame	<i>X</i>	<i>Y</i>	<i>Z</i>
Carolus (= REP-103 <code>_optical</code>)	right	down	forward
Intel D455 datasheet diagram	right	up	backward
SVGS (original NASA figure)	left	down	—
REP-103 <i>body</i> (the TF target)	forward	left	up

The D455 diagram differs from Carolus by a 180° rotation about *X* (*Y* and *Z* both negated); the SVGS figure differs by a 180° rotation about *Y* (*X* and *Z* both negated). Both are proper rotations, so both are legitimate right-handed frames — they are simply not this one.

Carolus (= REP-103 "_optical")

Intel D455 slide (NOT Carolus)

SVGS figure (NOT Carolus)

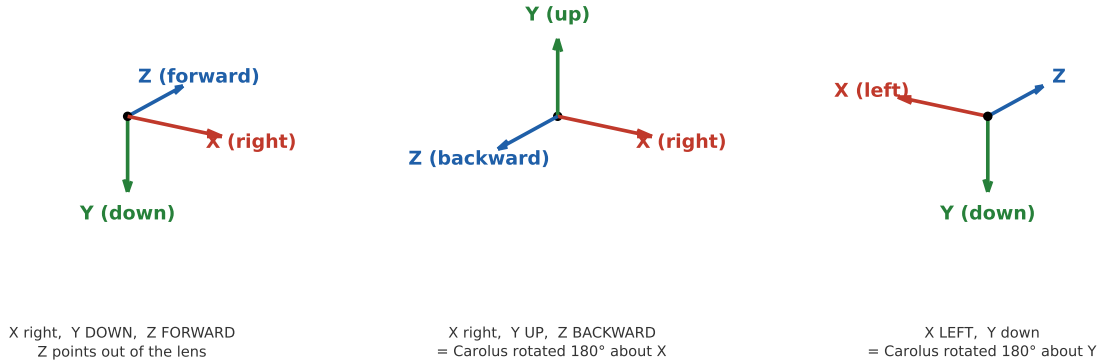


Figure 10.1: Carolus’s camera frame (left) against the two conventions most often confused with it. Only the left-hand frame is what `/pose` uses.

Roll, pitch and yaw are named after axes, so they do not survive a frame change either. The D455 diagram labels roll about X , pitch about Y , yaw about Z — the same *names* as REP-103 uses, attached to *different physical directions*, because its Y points up where Carolus’s points down. A “yaw” from one frame is not a “yaw” in the other. This is why Section 10.6.2 insists the conversion be applied to q_{ROS} and never to the raw `/pose` quaternion: the angles are only meaningful once the frame is.

10.4.3 The change-of-basis matrix

The remaining task is to re-express that rotation from Carolus’s camera axes (X right, Y down, Z forward) in ROS’s body axes (X forward, Y left, Z up). Writing the correspondence as a matrix acting on coordinate columns:

$$B = \begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}, \quad \det B = +1.$$

Both frames are right-handed, so B is necessarily a proper rotation. An improper (determinant -1) matrix here would assert that one of the two frames is a mirror image of what its own convention says, which is not the case for either.

10.5 Step 4 — a rotation is re-based by conjugation, not permutation

This is the step both prior integrations had to find empirically. A *vector* changes basis by a single multiplication, $\mathbf{v}' = B\mathbf{v}$. A *rotation operator* does not: it must be conjugated,

$$R' = B R B^{-1}, \quad \text{equivalently} \quad q' = b \otimes q \otimes b^{-1},$$

where b is the unit quaternion corresponding to B . Permuting and negating the components of q directly — the tempting shortcut, because it is what works for a vector — is not the same operation and does not generally give the same result.

The quaternion form $q' = b \otimes q \otimes b^{-1}$ exists only when B is proper. Unit quaternions parametrise proper rotations and nothing else, so an improper change of basis has no quaternion representation at all and must be handled as a matrix. This is another reason to establish the determinant before reaching for quaternions.

10.6 The result

Composing Steps 1–4, with q the quaternion read from `/pose`:

$$q_{\text{ROS}} = b \otimes q^{-1} \otimes b^{-1}$$

and the position, which is an ordinary vector and so takes the single multiplication:

$$(x, y, z)_{\text{ROS}} = (-z, x, -y)_{\text{camera}}.$$

In the broadcaster this is a few lines: build b once from B at import time, invert the incoming quaternion, conjugate, normalise, publish.

10.6.1 The numeric value of b , and how it is obtained

b is a constant. For the B of Step 3 it is, in (w, x, y, z) order:

$$b = \left(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, -\frac{1}{2}\right)$$

exactly — every component is $\pm\frac{1}{2}$, and $\|b\| = 1$. Geometrically it is a rotation of exactly 120° about the axis $\frac{1}{\sqrt{3}}(-1, -1, 1)$. **Verified** by computing it from B with the algorithm below and independently with `scipy.spatial.transform.Rotation.from_matrix(B)`: both give the same value. Note $-b$ describes the same rotation, so a library returning $(\frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}, \frac{1}{2})$ is not disagreeing.

Because every component is $\pm\frac{1}{2}$, the trace of B is 0 and the naive $w = \frac{1}{2}\sqrt{1 + \text{tr } B}$ shortcut divides by zero at the next step. The general method that handles this is **Shepperd's**: compute the trace, and if it is not positive, branch on the largest diagonal element instead.

```

1 import numpy as np
2
3 def matrix_to_quaternion(R):
4     """Shepperd's method. Returns (w, x, y, z), norm 1.
5     Branching on the largest diagonal element is what makes this
6     safe for matrices like B whose trace is 0 or negative."""
7     t = np.trace(R)
8     if t > 0:
9         w = 0.5 * np.sqrt(1.0 + t)
10        s = 1.0 / (4.0 * w)
11        x, y, z = (R[2,1]-R[1,2])*s, (R[0,2]-R[2,0])*s, (R[1,0]-R[0,1])*s
12    else:
13        i = int(np.argmax(np.diag(R)))
14        if i == 0:
15            x = 0.5 * np.sqrt(1.0 + R[0,0] - R[1,1] - R[2,2]); s = 1.0/(4.0*x)
16            w, y, z = (R[2,1]-R[1,2])*s, (R[0,1]+R[1,0])*s, (R[0,2]+R[2,0])*s
17        elif i == 1:
18            y = 0.5 * np.sqrt(1.0 - R[0,0] + R[1,1] - R[2,2]); s = 1.0/(4.0*y)
19            w, x, z = (R[0,2]-R[2,0])*s, (R[0,1]+R[1,0])*s, (R[1,2]+R[2,1])*s
20        else:

```

```

21         z = 0.5 * np.sqrt(1.0 - R[0,0] - R[1,1] + R[2,2]); s = 1.0/(4.0*z)
22         w, x, y = (R[1,0]-R[0,1])*s, (R[0,2]+R[2,0])*s, (R[1,2]+R[2,1])*s
23     return np.array([w, x, y, z])
24
25 B = np.array([[0, 0, -1],
26               [1, 0,  0],
27               [0, -1,  0]], dtype=float)
28 assert np.isclose(np.linalg.det(B), 1.0)          # proper rotation, Step 3
29 b = matrix_to_quaternion(B)                      # -> [-0.5, 0.5, 0.5, -0.5]

```

You do not have to trust the derivation. B is a constant, so b can simply be hardcoded — but check it, in whichever order your library uses. In `tf.transformations` and `geometry_msgs` the order is (x, y, z, w) , so the same value is written `[0.5, 0.5, -0.5, -0.5]`. Getting the order wrong produces a perfectly valid unit quaternion describing a different rotation, with nothing to signal it — the failure mode this whole chapter exists to prevent.

10.6.2 Converting the result to roll, pitch and yaw

Apply this to q_{ROS} , never to the raw `/pose` quaternion. Roll/pitch/yaw are only meaningful once the rotation is expressed in ROS body axes; running the conversion on the camera-frame quaternion produces three numbers that look reasonable and mean nothing.

REP-103 specifies **fixed-axis roll, pitch, yaw about X, Y, Z** respectively — equivalently intrinsic Z - Y - X , which is the same thing read in the other direction. With $q_{\text{ROS}} = (w, x, y, z)$:

$$\begin{aligned}\phi \text{ (roll)} &= \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2)) \\ \theta \text{ (pitch)} &= \arcsin(2(wy - zx)) \\ \psi \text{ (yaw)} &= \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2))\end{aligned}$$

Clamp the arcsin argument to $[-1, 1]$ before the call: round-off can push it just past 1 and return `nan`. At $\theta = \pm 90^\circ$ the decomposition is singular (gimbal lock) — roll and yaw stop being separable and only their sum is defined. The code below handles both.

```

1  import numpy as np
2
3  def quaternion_to_rpy(q_wxyz):
4      """REP-103 fixed-axis roll/pitch/yaw, radians.
5      Input MUST already be q_ROS, not the raw /pose quaternion."""
6      w, x, y, z = np.asarray(q_wxyz, float) / np.linalg.norm(q_wxyz)
7      sin_pitch = np.clip(2.0*(w*y - z*x), -1.0, 1.0)
8      if abs(sin_pitch) >= 0.999999:          # gimbal lock
9          pitch = np.copysign(np.pi/2, sin_pitch)
10         roll = 0.0                          # only roll+yaw is defined;
11                                         # convention: fold into yaw
12         yaw = np.arctan2(2.0*(w*z - x*y), 1.0 - 2.0*(x*x + z*z))
13     else:
14         pitch = np.arcsin(sin_pitch)
15         roll = np.arctan2(2*(w*x + y*z), 1 - 2*(x*x + y*y))
16         yaw = np.arctan2(2*(w*z + x*y), 1 - 2*(y*y + z*z))
17     return roll, pitch, yaw

```

The gimbal-lock branch is not the general formula with θ substituted, and getting it wrong is easy to miss. Its `atan2` arguments are $2(wz - xy)$ and $1 - 2(x^2 + z^2)$ — note the *minus*, and $x^2 + z^2$ rather than $y^2 + z^2$. Both differ from the general yaw expression above. A first draft of this section used `2atan2(x, w)` here; it is wrong, and it is wrong only within about 0.3° of $\theta = \pm 90^\circ$, so casual testing will not catch it.

If `tf.transformations` is already imported, it ships the same conversion and is the shorter route — note the argument order is (x, y, z, w) and the return order is (roll, pitch, yaw):

```
1 from tf.transformations import euler_from_quaternion
2 roll, pitch, yaw = euler_from_quaternion([x, y, z, w])    # xyzw in, rpy out
```

Verified, 2026-08-25, by reconstructing a rotation matrix from the returned angles and comparing it against the original:

Check	Max. matrix difference
5000 random rotations, general branch	1.2×10^{-14}
1250 gimbal-lock cases, both signs	5.6×10^{-16}
q_{ROS} chain vs $B R^\top B^{-1}$, 2000 quaternions	6.7×10^{-16}

Within the lock band the reconstruction differs by $O(\varepsilon)$, where ε is the angular distance from exactly 90° — that is the decomposition’s own ambiguity, not an error in the code.

10.7 Cross-checking the result against an independent implementation

A derivation that only reproduces its own assumptions is worth little. The Limo integration mentioned earlier arrived, by trial and error and with no derivation, at a component-wise rule for its own quaternion conversion, using the same axis correspondence B as above.

Evaluating both expressions on randomly generated unit quaternions gives identical rotations, agreeing to machine precision (differences of order 10^{-16} , i.e. floating-point round-off) across every trial. The formula derived here therefore *predicts* an empirical result obtained independently, on different hardware, by people who were not deriving it — which is the strongest available evidence that the derivation is not merely self-consistent.

Applying the same comparison to the pre-correction formula (composition without the inversion of Step 2) gives a substantially different rotation on every trial, confirming the inversion is load-bearing rather than cosmetic.

10.8 Validation status

Verified: the derivation above, checked against the solver’s own source; the determinant argument, checked against REP-103’s stated axis conventions; and the numerical agreement with the independent Limo result, computed directly.

Verified end-to-end on hardware, 2026-08-17. The beacon was rotated by roughly 30° about each camera axis in turn, held still, and `tf_echo camera_link beacon_observed` read before and after — bracketed by a second reference reading once the beacon was returned to straight, so the small residual attitude drift documented in Section 14.4 could be measured and subtracted rather than assumed negligible.

Axis	Gesture	Measured (drift-corrected)
Yaw (Z)	tripod pan, $\sim 30^\circ$ clockwise from above	-28.8°
Pitch (Y)	tripod tilt, $\sim 30^\circ$ backward	-23.7°
Roll (X)	hand twist, $\sim 30^\circ$, steering-wheel motion	-23.6°

Zero axes inverted. Each rotation landed on its own axis — the other two moved by at most a few degrees, the residual of an imperfect hand gesture rather than cross-talk between axes — and every measured sign matches the derivation above. **The 2026-08-11 formula is confirmed on this robot, not just on paper.**

One methodological note worth keeping: **the first attempt at the roll axis was itself invalid, and it is worth knowing why.** A rigid tripod's own rotation is a vertical pan, which is yaw, not roll — a rotation about the camera-to-beacon line needs the beacon twisted by hand, independent of any tripod mechanism. The two are easy to conflate verbally ("spin the beacon") while being mechanically distinct, and the first attempt reproduced the yaw result almost exactly under the roll label before this was caught by comparing the two readings side by side.

Chapter 11

Operating the Robot: the Launcher

Runs on: the **lab PC** — the launcher is a Tkinter GUI that starts everything else over SSH. The robot and the Pi must both be on.

Everything up to here produced a pose on `/pose`. This chapter is how the robot is actually driven. The tool is `shortcuts/carolus_launcher.py`, in the repository you cloned; it starts the whole stack with no commands to type, shows a live dashboard, and pilots the robot from the keyboard.

```
python3 shortcuts/carolus_launcher.py
```

Fuller reference: `shortcuts/README.md` in the repository documents every button, panel and keyboard binding, and is kept current with the code. This chapter covers what a first-time operator needs.

11.1 The launch tabs, in order

Each tab is one process. They are started in order — the first two wait for a concrete signal before unlocking the next, rather than a fixed delay.

Tab	What it starts	Unlocks when
T1 roscore [Pi]	SSH: <code>eth1 up + roscore</code> on the Pi	port 11311 is open (60s timeout)
T2 Camera [Pi + PC]	SSH: <code>rm_cam_beacon.py</code> on the Pi. Then, <i>locally on the lab PC</i> , <code>cam_view_helper.py</code> — not over SSH (Section 14.2)	<code>/camera/color/image_raw</code> is publishing (60s timeout)
T3 Carolus [Pi]	SSH: <code>roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false</code>	manual — no wait
T4 TF [Pi]	SSH: <code>carolus_tf_broadcaster.py</code>	manual. Republishes <code>/pose</code> as a TF (<code>camera_link</code> → <code>beacon_observed</code>). Not needed by the SEARCH/ALIGN/APPROACH pipeline, which reads <code>/pose</code> directly
T5 MINS [Pi]	SSH: MINS's own <code>simulation.launch</code> , in a separate sandbox workspace (Chapter 12)	independent — runs its own master, unrelated to the pipeline above. Not killed by KILL ALL, for that reason; use its own Kill button

T1–T3 are the working stack. T4 is for validating orientation and for future sensor-fusion work; T5 is unrelated experimental work. A normal session is T1, T2, T3.

Every tab label names its machine (T1 roscore [Pi], T5 MINS [Pi]). This is deliberate: the ROS master lives on the Pi in every configuration, so nothing else on screen distinguishes a node running on the Pi from one running on the workstation — and a rate measurement was once attributed to the wrong machine for exactly that reason.

11.2 Driving the robot by hand

The launcher is MANUAL-only. It sends `MODE MANUAL` once T2 reports the camera ready, and there is no mode switch — `MANUAL` is also the safe default, being the mode in which the robot never moves on its own. Two key blocks light up as keys are pressed.

Chassis — ZQSD	Gimbal — numpad
Z forward S reverse	8 pitch up 2 pitch down
Q rotate left D rotate right	4 yaw left 6 yaw right 5 stop
0.20 m/s linear, 20°/s angular. Stops automatically when keys are released.	30°/s pitch, 40°/s yaw. Works with Num-Lock on or off.

Clicking `MODE` again returns to `AUTO` and stops both immediately.

If the keys stop responding, move the mouse over the launcher window. T1 and T3 can open external terminal windows that steal the system’s keyboard focus. Hovering over the launcher reclaims it automatically. This is not a bug in the robot — the commands are simply not reaching the application.

The numpad does nothing while LOCK is on. LOCK takes exclusive control of the gimbal; the chassis keys keep working normally. Turn LOCK off to get manual gimbal control back.

11.3 The main controls

Control	What it does
LOCK	Periodically re-centres the gimbal on the beacon. Period configurable in seconds. Ignores corrections below a 5° deadband and skips any beyond 45° (an implausible pose).
CAM PREVIEW	Toggles the camera view. Off by default — it costs network bandwidth and GUI smoothness, so turn it on only when you need to see the image.
BLOB VIEW	Toggles a second panel showing <code>/postprocessed/image</code> — Carolus’s own black-and-white thresholded view, with a coloured circle on each blob it kept after filtering (Section 9’s detection parameters). Off by default , same reasoning as CAM PREVIEW. Added 2026-08-14, this is the modern equivalent of viewing the same topic through rviz — useful for confirming only the beacon’s 4 LEDs are being picked up, not a reflection or another light source.
Kill	Stops everything and <i>verifies</i> the remote processes actually died. KILL ALL covers T1–T4 but deliberately not T5 (MINS), which is independent — kill that one from its own button.
F11 / Escape	Fullscreen on / off.

The **dashboard** is split into two panels, **Robot Status** (state, beacon distance, battery, attitude, chassis speed, wheel RPM) directly above **Raspberry Pi Status** (the Pi’s own temperature, load and RAM, refreshed every 20s, separated out 2026-08-14 since it describes the Pi carrying the perception pipeline, not the robot itself). The Pi’s temperature turns amber at 65°C and red at 75°C , ahead of the Pi 4B’s 80°C thermal throttle.

The **camera and blob-detection panels resize with the window**, added 2026-08-14: both grow together, in whole multiples of their base 480×270 , up to $3\times$, as the launcher window is enlarged — enlarging the window is therefore the way to get a bigger view, there is no separate zoom control. The underlying image transferred from the Pi stays 480×270 regardless; only the on-screen display size changes, so this costs no extra network bandwidth.

The **beacon indicator** reads BEACON: DETECTED (green) or BEACON: LOST (red). To see *where* the beacon sits within the frame, use CAM PREVIEW or BLOB VIEW.

Beside it, the **beacon signature** shows the saturated-pixel count and hue of what the camera is actually seeing, judged against the recorded reference in `carolus_node/config/beacon_reference.yaml`: a plain green reading, or one of TOO BRIGHT (blobs merge), hue OUT OF WINDOW, NO LIGHT. **Read this before diagnosing any detection failure** — as Section 9.9 explains, an over-bright beacon reports itself in the logs as an *absent* one. It is computed inside the camera preview’s own callback, so it only updates while CAM PREVIEW is ON.

11.4 Session logs on disk

Every line shown in any tab is also written to `shortcuts/logs/session-YYYY-MM-DD-HH-MM-SS.log`, one file per launcher start, each line tagged with its tab. The on-screen tabs are truncated to 300 lines and lost on exit, so the file is the only durable record — and it makes questions answerable after the fact:

```
grep '\[T2\].*\[LOCK\]' shortcuts/logs/session-*.log
grep '\[T3\].*\[LATENCY\]' shortcuts/logs/session-*.log
```

11.5 Shutting down, and why it matters

Use the launcher's Kill, or close the window — both clean up the remote processes and re-read the Pi to confirm each one actually died, logging `TOUJOURS VIVANT` for any survivor rather than failing silently.

An orphaned `rm_cam_beacon.py` breaks the next session. It keeps the robot's single SDK connection open, so the next launch hits the double-connection failure — motion commands stop responding with no error anywhere. If you ever kill processes by hand, **bracket the pattern:** `pkill -f "[r]m_cam_beacon"`. Without the bracket, the remote shell's own command line contains the pattern and `pkill` can kill that shell before it reaches the real target, non-deterministically and with nothing reported.

11.6 Motion commands from Carolus readings: beacon docking (removed from the launcher)

This is the project's motion-command-generation code, in case you are searching for it by that description rather than by “docking” — it drives the chassis and gimbal from Carolus's own /pose readings, which is exactly what that phrase means. **Not reachable from the GUI since 2026-08-14.** The docking behaviour had its own launcher tab and control panel; both were removed at the project's request, along with the `/carolus/dock` relay in `cam_view_helper.py` that carried its commands. **The script itself, `carolus_ws/src/robomaster_cam/scripts/beacon_docking.py`, is still in the repository and still works** — it is simply no longer wired to a button. To use it you would run it by hand and publish to `/carolus/dock` directly (`rostopic pub`); its own `-selftest` flag runs its pure-geometry suite with no robot needed.

What it does, for anyone who picks it up again: drives the robot to a fixed distance from the beacon (0.20 m) in three phases — align the gimbal to the beacon, rotate the chassis until it agrees with the gimbal, then drive forward. Commands are `CALIBRATE` and `CALSTEP2` (a two-step calibration: face-on, then again after *you* have pivoted the robot roughly 30–45°), then `START`, with `ABORT` to stop.

Two known limitations, both unresolved, if you re-enable it. Nothing corrects the chassis heading *during* the drive — only the camera tracks the beacon, so a run can finish close to the beacon but off-centre. A collision happened once on this project, caused by a silent gimbal-probe failure that let the chassis align to the wrong reference (since fixed, with a retry and an abort guard). And it requires `RM_ROBOT_MODE=free` — in `chassis_lead` its chassis alignment has no observable to converge on (Section 14.3), which the script now refuses outright rather than failing obscurely. **Run docking with clear space around the robot, and keep a hand near ABORT.**

Chapter 12

MINS on the Ground Station

Runs on: the lab PC, in its own isolated workspace — `mins_sandbox/catkin_ws`, deliberately separate from `carolus_ws`. Nothing here is required to run the robot itself; this chapter is for fusing a recorded session’s camera and IMU data through MINS (github.com/rpng/open_vins’s multi-sensor successor) as a ground-station replay, not for anything that runs live on the robot.

12.1 What this is, and what it is not

MINS is not part of the robot’s own pipeline — Carolus already produces a pose from the beacon, and nothing here changes that. This chapter is a *separate* estimator, replayed offline against a recorded rosbag, to compare against Carolus/the wheel odometry on the same trajectory. Two consequences follow directly:

- It needs no robot connection to build or to run its own bundled synthetic demo (Section 12.2–12.3). It *does* need a real recorded rosbag, from the robot, to do anything useful with this project’s own data (Section 12.5).
- `mins_sandbox/catkin_ws` is intentionally not `carolus_ws`. Building or rebuilding anything in this chapter never touches the packages the robot itself depends on.

12.2 Building on the lab PC (Ubuntu 22.04)

Verified 2026-08-11/2026-08-13. This machine’s Ubuntu 22.04 ships native library versions (OpenCV 4.5.4, `log4cxx` 0.12, GDAL 30/PROJ 22) newer than what ROS Noetic’s prebuilt Ubuntu-20.04-targeted binaries were built against (OpenCV 4.2, `log4cxx` 0.10, GDAL 26/PROJ 15). Every build failure below traces back to the same root cause — two incompatible major versions of one C++ library loaded into the same process — appearing in three different places. **The Raspberry Pi runs Ubuntu 20.04 natively, ROS Noetic’s actual target, so none of this class of bug is expected to recur there** (checked directly for the `log4cxx` case: only one version installed on the Pi).

1. Clone the workspace’s three source trees into `mins_sandbox/catkin_ws/src/`: `mins` (github.com/rpng/open_vins’s MINS fork — itself a metapackage bundling `ov_core`, `ov_init`, `ov_msckf`, `ov_eval`, `mins`, `mins_eval`, `mins_data` and its `BSplineDataset/FloorPlans` synthetic datasets), `vision_opencv` (`ros-perception/vision_opencv`, the noetic branch), and `image_transport_plugins` (`ros-perception/image_transport_plugins`, the noetic-devel branch — confirmed with `git ls-remote -heads` rather than assumed, `vision_opencv`’s branch name does not carry over).

2. **Fix 1 — cv_bridge/OpenCV ABI mixing.** The Debian-packaged `ros-noetic-cv-bridge` links against OpenCV 4.2; this workspace's own code resolves to the one real OpenCV install on this machine, 4.5.4. Mixing the two crashes at process exit inside `libproj` (two `libproj` major versions loaded simultaneously, pulled in transitively through each OpenCV build's GDAL support) — confirmed with a standalone `gdb` reproducer built from real camera frames, not inferred from a warning. **Fix:** build `cv_bridge/image_geometry` from the cloned `vision_opencv` instead of using the Debian package, so everything in the workspace resolves to one OpenCV (4.5.4). One extra link fix needed on the way: `cv_bridge.cpp` calls `boost::regex` but its own `CMakeLists.txt` never linked `Boost_LIBRARIES` — add `boost_regex` explicitly.
3. **Fix 2 — ov_core/ov_eval need C++17, not C++14.** `liblog4cxx-dev` 0.12.1-4 on this machine typedefs `std::shared_mutex` in a header `ros/console.h` pulls in unconditionally, which needs C++17; `mins/mins_eval` already requested it, `ov_core/ov_eval` did not. Bump both.
4. **Fix 3 — stale catkin_LIBRARIES entries.** `cv_bridge`'s re-exported link list references OpenCV modules (`objdetect`, `contrib`, `photo`, `shape`, `stitching`, `superres`, `video`, `videostab`, `viz`) that do not exist as *4.2 packages on this system. Rather than blacklisting module names one failed link at a time, filter `catkin_LIBRARIES` in `ov_core/ov_eval/ mins/mins_eval`'s `CMakeLists.txt/ cmake/ROS1.cmake` to drop any entry that is an absolute path to a file that does not exist on disk.
5. **Fix 4 — liblog4cxx 10-vs-12 collision.** `mins_lib.so` ended up linked against both `liblog4cxx.so.10` (via ROS's own prebuilt `librosconsole_log4cxx.so`, a hardcoded `NEEDED` entry) and `liblog4cxx.so.12` (this Ubuntu 22.04 system's native package), through an ambiguous `liblog4cxx.so` symlink that Fix 3's filter did not catch (the file exists on disk, it just resolves to the wrong major version). Confirmed with `gdb`: `SIGSEGV` inside `liblog4cxx.so.12`'s destructor, called from code built against `.so.10`. **Fix:** extend the same filter with a name match on the ambiguous `liblog4cxx\.$` symlink specifically, leaving `.so.10` to arrive exactly once, transitively, via `roscpp`.
6. **Fix 5 — compressed_image_transport ABI mixing.** A second, later `SIGSEGV` in `libproj`, this time invisible to a static `ldd` walk of the built executable — it comes from a `pluginlib/dlopen` load at runtime, not a link-time dependency. The prebuilt Noetic `.deb` for `compressed_image_transport` links the old OpenCV 4.2/GDAL 26/PROJ 15 chain, colliding with this workspace's own 4.5.4/GDAL 30/PROJ 22 stack the moment `image_transport` advertises a publisher and `pluginlib` loads every registered transport plugin. **Fix:** rebuild all three plugins in the cloned `image_transport_plugins` from source — `compressed_image_transport`, `compressed_depth_image_transport` and `theora_image_transport` (the first fix attempt rebuilt only `compressed_image_transport` and the crash did not go away — `compressed_depth_image_transport` turned out to be the plugin actually being loaded, confirmed with `info sharedlibrary` inside `gdb` rather than assumed from the first fix looking plausible).
7. Build the whole workspace: `catkin build` from `mins_sandbox/catkin_ws`. All packages succeed; `devel/lib/mins/` contains three binaries — `simulation`, `bag`, and `subscribe`.

Known non-blocking issue. After Fix 5, a different exception appears at process teardown — `terminate` called after throwing an instance of `'class_loader::LibraryUnloadException'` — thrown from `class_loader`'s own book-keeping while unloading the locally rebuilt `compressed_depth_image_transport`, not an ABI conflict. It fires strictly at `exit()`, after all output files are already written correctly (verified: the trajectory file and its timing file are present and correct before the exception fires, both in this workspace and, independently, on a Pi build of the related `sqrtVINS` project using the stock unmodified Noetic `.deb` — the same exception class on a machine with no local rebuilds at all, so this is a general `pluginlib/class_loader` quirk in this ROS Noetic era, not something this workspace's rebuild introduced). Confirmed harmless by inspecting the output, not by ignoring the crash.

12.3 Running the bundled simulation (proof of life)

Verified 2026-08-13. MINS ships its own self-contained synthetic dataset — no download, no robot needed.

```
source /opt/ros/noetic/setup.bash
source mins_sandbox/catkin_ws/devel/setup.bash
roslaunch mins simulation.launch
```

The `dataset` argument defaults to `UD_Warehouse` (other bundled options: `EUROC_MH1`, `EUROC_V11`, `TUM_C`, `TUM_R1`, `UD_Gore`, `UD_Small`, `UD_Spencer` — see `mins/mins/launch/simulation.launch` for the full list and override with `dataset:=NAME`). A successful run writes, under `outputs/tmp/`: `0.txt` (a TUM-format trajectory — one line per pose, `timestamp x y z qx qy qz qw` — with full covariance), `0/imu_gt.txt/imu_est.txt/imu_std.txt` (ground-truth vs. estimated vs. propagated-uncertainty IMU-frame pose), and `0.time` (the run's own wall-clock duration). **Check these files exist and look physically plausible before trusting a run** — the teardown exception noted above means the process can exit with a visible exception even on a successful estimation.

12.4 Config and launch file reference

File	Role
<code>mins/mins/launch/simulation.launch</code>	Entry point for the bundled synthetic demo (Section 12.3).
<code>devel/lib/mins/bag</code>	The replay entry point for a real recorded rosbag (Section 12.5) — confirmed present, not yet exercised against this project's own data.
<code>devel/lib/mins/subscribe</code>	Live-subscribe entry point (consumes ROS topics directly instead of a bag) — present, not evaluated for this project; replay was confirmed sufficient for the ground-station use case (Hector explicitly accepts rosbag collection over a live run).
<code>mins_data</code> (bundled package)	<code>BSplineDataset/FloorPlans/</code> — the synthetic trajectories/floorplans <code>simulation.launch</code> reads via <code>\$(find mins_data)</code> .

12.5 Recording a rosbag for a real replay

Not yet executed — a procedure to follow, not a report of a run that happened. Written from the exact commands `plan-fin-de-stage.md` specifies and from reading `ov_eval`'s own alignment code (`perform_association`), not from having done it. The honesty markers on this section are **Inherited/not executed** throughout.

MINS needs a camera topic and an IMU topic recorded *simultaneously* in one bag — checked across all 26 `.bag` files anywhere in this project (including `old//documentations/`) and none carries both at once, so this recording does not yet exist anywhere.

1. Lay out a rectangle on the floor with tape; measure and record the *actual* corner-to-corner lengths (not nominal ones).
2. Start recording both `/camera/color/image_raw` and `/imu` (both already publish independently, confirmed working on the Pi — this is the first session to capture them into the same file):

```
rosvarecord /camera/color/image_raw /imu -O rectangle_run.bag
```

3. Drive the robot manually (ZQSD, the launcher's MANUAL mode) around the taped rectangle. Manual piloting is explicitly sufficient here — Hector's own ask does not require autonomous motion to work reliably.
4. **Mark the exact moment the robot passes each corner** — `ov_eval`'s trajectory alignment needs at least 3 common timestamps between the ground truth and the estimate to align at all. A coordinate alone, without its timestamp, cannot be used.
5. Before trusting the bag: re-run `shortcuts/tear_check.py` (the torn-frame detector, already validated 6/6 against synthetic seams and confirmed 0/60 under chassis motion in an earlier session) immediately before the real take, not as a standing clearance from a previous session — the torn-frame defect it screens for is intermittent.

12.6 Plotting the estimate against ground truth

Not yet executed, same honesty marker as Section 12.5 — this is `ov_eval`'s documented usage, confirmed by reading `plot_trajectories.cpp`, not a report of having run it.

1. Replay the rosbag through MINS: `devel/lib/mins/bag`, pointed at `rectangle_run.bag` (exact CLI arguments to be confirmed against bag's own `-help` at the point this is first run — not assumed here).
2. Extract the live pose topic to a TUM-format file:

```
rosvarecord ov_eval pose_to_file <pose_topic> estimate.txt
```

3. Build a sparse ground-truth TUM file by hand from the taped rectangle's four measured corner coordinates and their logged pass-through timestamps (Section 12.5, step 4) — `timestamp x y z qx qy qz qw` per line, orientation columns can be the identity quaternion if corner heading was not recorded.
4. Align and plot:

```
roslaunch ov_eval plot_trajectories <align_mode> ground_truth.txt estimate.  
txt
```

`ov_eval`'s Absolute Trajectory Error alignment implements Zhang & Scaramuzza, IROS 2018 (documented by OpenVINS itself) — cite this rather than describing the alignment as unexplained. No prior tutorial or repository was found for the sparse, hand-measured (taped-corner) ground-truth method itself specifically — this is original protocol design for this project, not a known pattern to fall back on if the result looks wrong.

12.7 sqrtVINS — the comparison partner

A separate, related project (github.com/rpng/sqrtVINS, an OpenVINS-family square-root-filter estimator built for embedded hardware, no wheel-encoder input) is being evaluated *on the Pi*, in its own isolated workspace (`~/sqrtvins_ws`, not this chapter's `mins_sandbox`), as the platform this ground-station MINS build is meant to be compared against.

```
mkdir -p ~/sqrtvins_ws/src && cd ~/sqrtvins_ws/src  
git clone https://github.com/rpng/sqrtVINS.git  
cd .. && catkin build ov_srvins -j1  
catkin build
```

The default parallel build OOM-kills on this Pi. `catkin build`'s default 4-way parallelism ran out of memory compiling `ov_srvins` — this Pi has no swap configured, confirmed via `dmesg`. Build that one package alone at `-j1` first, then the rest of the workspace normally; a plain `catkin build` with no flags will very likely repeat the OOM.

Verified 2026-08-13: builds cleanly on the Pi with the flags above, and its own bundled simulation produces genuine trajectory output there too. The real comparison — update rate and accuracy against this chapter's MINS, over the *same* rectangular-motion trajectory — needs Section 12.5's recording to exist first, and is not yet done.

Chapter 13

Carolus with a USB Webcam

Runs on: the lab PC (or any ground-station machine), with a USB webcam. No robot, no Pi, no RNDIS connection — this chapter is for running Carolus’s detection and pose pipeline standalone, against a beacon held or mounted in front of an ordinary webcam, for calibration and detection-tuning work off the robot.

13.1 Why this exists

Carolus’s own architecture already supports this without modification: `carolus.launch` exposes `camera_topic` and `config` as real arguments with defaults, precisely so the pipeline can run against any image source once it is told the topic name and given a matching hardware profile. `testcarolus.launch`, the robot’s own entry point, hardcodes both to the RoboMaster’s values — this chapter’s entry point, `carolus_logitech.launch`, is a sibling to it, not an edit to it.

13.2 Installing the camera driver

`usb_cam` provides the ROS driver. **On Ubuntu 20.04, the prebuilt package works:**

```
sudo apt install ros-noetic-usb-cam
```

On Ubuntu 22.04, that package does not exist, and cannot — ROS Noetic never had official binary packages past Ubuntu 20.04. Build from source instead:

```
cd carolus_ws/src
git clone --depth 1 -b master https://github.com/ros-drivers/usb_cam.git
cd ../../ && catkin_make --pkg usb_cam
```

`master`, not `main` — the latter is the ROS2 branch. This project’s own copy in `carolus_ws/src/usb_cam/` already carries three fixes a stock checkout needs on a modern system: renamed FFmpeg API calls, and a `CMakeLists.txt` filter removing an ambiguous `liblog4cxx.so` reference that otherwise crashes the node at startup on any machine where that symlink resolves to a newer major version than ROS Noetic’s own prebuilt console backend expects (exit code -11, every launch). If `catkin_make` reports a missing `libavcodec.so` at the link step, that specific file is missing from an otherwise-installed `libavcodec-dev` on at least one observed machine — `sudo apt install -reinstall libavcodec-dev` resolves it.

13.3 Running it

```
roslaunch usb_cam usb_cam-test.launch
```

Publishes the raw feed on `/usb_cam/image_raw`. **Verified 2026-08-17:** 30.0 Hz average over a 284-sample window, std dev 0.0018 s, on a Logitech HD Pro Webcam C920.

13.4 The Carolus config

`carolus_node/config/logitech_1080p.yaml` follows `robomaster_s1.yaml`'s own documented pattern: copy it, replace the camera intrinsics and distortion with your own camera's, keep the beacon geometry if it is the same physical beacon. **Distortion must be a real YAML list** (`[k1, k2, p1, p2]`, four values), not a quoted string — `getParam` into a `vector<double>` silently fails to load a string and falls back to a hardcoded placeholder with no warning, which is exactly what happened to this robot's own profile until it was found and fixed (see `robomaster_s1.yaml`'s own inline comment on the `distortion` field for the full account).

```
roslaunch carolus_node carolus_logitech.launch
```

Verified 2026-08-17: run against the live 30 Hz feed above, no crash, every frame processed. **That is a startup check, not a detection check** — it says the node consumed frames, not that it found a beacon in any of them. See the warning below before concluding anything from a clean-looking run.

image_threshold must be 100 here, not the RoboMaster's 190 — corrected 2026-08-21. This profile was written by copying `robomaster_s1.yaml` and replacing the intrinsics; `image_threshold` came along unexamined. At 190 a blue-LED beacon is *never detected*, and nothing says so: `preprocessImage` applies `cv::threshold(..., image_threshold, 255, THRESH_BINARY)` first, so the frame goes black before any colour test runs, `findContours` returns nothing, and the log reads `0 contours found`. The node stays up, the topics look healthy, and no pose is ever published.

The cause is physical rather than a property of one camera: a saturated blue pixel carries little weight in the standard $0.299R + 0.587G + 0.114B$ grayscale conversion, so blue LEDs that are plainly bright to the eye land low in grayscale. **Measured** on a Logitech C920 with the beacon lit and in frame (2026-08-20, lab PC; reconfirmed 2026-08-21, Raspberry Pi 5): hue 105–115 and saturation 200–255, both comfortably inside their windows, but grayscale peaking at **131**. Moving the beacon closer with the threshold left at 190 still gave zero contours — which is what rules out blob size and distance and leaves the luminance ceiling. For a different camera or beacon, measure rather than copy: the threshold must sit below the observed grayscale peak.

13.5 Reading the result

`/postprocessed/image` — the black-and-white blob view — is **only published on a successful four-blob detection**. Silence on that topic by itself does not mean anything is broken: it means no valid beacon has been found in any frame since the node started. Before suspecting a bug, confirm a lit beacon is actually in frame; then check the terminal running `carolus_astrobees`. The two failure messages are not interchangeable, and they point at different parameters:

Log line	What to check, in this order
0 contours found	Nothing survived the brightness cut, so no colour test ever ran. <code>image_threshold</code> first — it must sit <i>below</i> the beacon’s grayscale peak (see the warning in the config section; 190 finds nothing on a blue-LED beacon). Then exposure and whether the beacon is actually lit.
Not enough blobs with required circularity	Contours were found and then rejected, so brightness is fine. Hue window (<code>lb_hue/ub_hue</code> , OpenCV’s 0–180 scale), <code>min_area/max_area</code> , <code>saturation_threshold</code> , <code>min_circularity</code> .

Either of those is a tuning problem. No output at all, or a crash, is a different and deeper problem.

`carolus_rviz.launch` (Chapter 11 covers its counterpart on the robot) displays both the raw feed and the blob view side by side. **Its saved config points the “Camera raw” panel at `/camera/color/image_raw`** — the RoboMaster’s topic name. Against this chapter’s setup, change that panel’s Image Topic field to `/usb_cam/image_raw` by hand; nothing publishes on the RoboMaster’s name here, which is why the panel would otherwise show “No Image” indefinitely.

Chapter 14

Troubleshooting

Runs on: wherever the symptom appears. Each entry names the machine to check on.

Work down from the symptom. Each step either fixes the problem or rules out a layer — the ordering matters, because a dead network link and a dead node look identical from most checks.

14.1 Nothing responds: the robot ignores every command

1. **Did the robot chime twice at power-on?** One chime means the root hack is not active and the SDK will not answer at all (Chapter 3). Power-cycle and listen again.
2. **Is a second script holding the SDK?** Only `rm_cam_beacon.py` may connect. On the Pi: `pgrep -af "[r]m_cam_beacon"` — more than one result, or an orphan from a previous session, causes exactly this symptom with no error message anywhere. Kill it and relaunch.
3. **Is the RNDIS link up?** From the Pi: `ping -c4 192.168.42.2`. It does not survive a Pi reboot (Chapter 6).

14.2 No camera preview, and the camera/gimbal buttons do nothing

These two symptoms always appear together, and they have one cause. `cam_view_helper.py` does two jobs at once: it writes the preview thumbnail the launcher displays, and it is the relay through which *every* camera and gimbal command reaches ROS — the numpad, LOCK, and manual driving. If it is not running, all of them are dropped.

It runs on the lab PC, not the Pi, as a local subprocess of the launcher (the launcher writes to its standard input, so the two must share a machine). Check it there:

```
pgrep -af "[c]am_view_helper"
```

Nothing returned means every camera and gimbal control is inert. Restart T2. If it dies again, its own error now appears in the T2 log tab prefixed [HELPER], and the launcher prints an explicit warning a few seconds after launch — read those lines rather than guessing.

A dead helper used to be invisible, which is worse than a crash. The launcher reported each button press as delivered whether or not it arrived. If you are working from an older copy of this software and see the GUI acknowledge commands that plainly have no effect, this is the first thing to check — the acknowledgement means the button was pressed, not that the robot heard it.

14.3 Choosing the robot mode: the camera wanders off target

Symptom. The robot is stationary, nothing is commanded, and yet the camera slowly turns away from whatever it was pointed at — losing the beacon after a few minutes. Left alone the gimbal keeps going, eventually reaching its mechanical limit.

Cause, established by measurement 2026-08-14. The robot’s own attitude estimate drifts — measured directly, with the gimbal servo taken out of the loop, at $+7.8^\circ/\text{min}$. The gimbal’s stabilisation loop holds a constant heading in the *power-on* frame (`gimbal_yaw_ground`), and that frame is derived from the drifting estimate. Holding a fixed heading against a moving reference means the firmware **physically rotates the gimbal**, continuously. The chassis does not move; the camera does.

Fix: a chassis-referenced hold, which leaves the gimbal aimable. The drift shows up in the *inertial* frame, but not in `gimbal_yaw_rel` — the gimbal-to-chassis joint angle, an encoder reading rather than an integrated estimate, so it cannot drift. The node therefore holds *that* constant instead: every 2s it compares the current `yaw_rel` against the angle the operator last asked for and issues one small relative move to close the gap. Driving the gimbal by hand simply moves the target, so manual aiming is unaffected. Deliberately a slow trim rather than a servo (2s period, 0.8° deadband, $60^\circ/\text{s}$), and it stands aside for LOCK, for an in-flight recentre, and while a manual command is live. Nothing to configure — `RM_GIMBAL_HOLD=off` disables it.

Measured (2026-08-14, stationary, 45–60 s windows):

Configuration	<code>gimbal_yaw_rel</code> drift
Torque on, no hold (the old behaviour)	$-6.35^\circ/\text{min}$
Torque cut entirely	$+0.00^\circ/\text{min}$, but unaimable
Torque on + chassis-referenced hold (default)	$-0.20^\circ/\text{min}$

Roughly a thirty-fold reduction with aiming intact: over a half-hour measurement, about 6° of movement instead of 190° . Verified in the same run that the gimbal still responds normally — commanded $+25^\circ/\text{s}$ for 1.5s it moved $+33.1^\circ$, and $-25^\circ/\text{s}$ moved it back -39.9° — before settling to the figure above.

Cutting the torque outright also works, and is the better choice for one specific case. `RM_GIMBAL_TORQUE=off` recentres the gimbal and then suspends it: absolutely motionless ($+0.00^\circ/\text{min}$, spanning 0.2° across 381 samples — the encoder’s own quantisation step), at the cost of a gimbal that cannot be aimed at all, with the numpad and LOCK inert. Worth it when a measurement needs the camera perfectly still and nothing else, not for ordinary operation. **The order matters if you use it:** suspending torque alone lets the gimbal fall to its rest position — measured, it dropped 17° and lost the beacon — so the node recentres first and only then cuts.

Correction to an earlier claim in this manual. Until 2026-08-14 this section said the fix was to run in `chassis_lead` rather than `free`, citing a drop to $-0.00007^\circ/\text{s}$. **That was wrong, and the number was measured with a blind instrument.** It was taken on `gimbal_yaw_rel` — the gimbal-to-chassis angle — which in `chassis_lead` is constant *by construction*, since the gimbal follows the chassis by definition of the mode. A flat reading there is compatible with the robot being perfectly still and with it spinning on the spot. Re-measured on the same signal with the same mode active, the drift is $-6.35^\circ/\text{min}$. The mode changes nothing; the torque does.

Still not a repair of the underlying defect. The attitude estimate is as wrong as it ever was — [ATTI] pitch, roll and yaw remain unusable as an absolute reference, and anything reading them accumulates roughly 5–8°/min of yaw error. What is fixed is that the gimbal no longer acts on it.

Docking needs RM_ROBOT_MODE=free, separately from all of the above: it aligns the chassis by driving /carolus/gimbal_yaw_rel to zero, and in chassis_lead that angle cannot change when the chassis turns, so the alignment never converges. The script refuses to start in the wrong mode rather than failing obscurely.

14.4 The robot slowly rotates on its own

A stationary robot that yaws a few degrees at a time, accumulating to tens of degrees over several minutes, with nothing commanding it. This invalidates any static measurement — a calibration or a pose comparison taken while the robot is quietly turning is worthless — so it is worth ruling out before trusting a session’s numbers.

Check the surface first, before reading any code. Investigated at length on 2026-08-12: the software was exonerated on five separate counts, and the one variable that tracked the symptom was what the robot was standing on. Mecanum wheels roll sideways by design; their angled rollers offer very little resistance to a slow yaw, so a smooth surface that is even slightly off-level lets the robot creep. **Measured 2026-08-12:** on a table the drift ran at roughly 0.1°/s; moved to the floor, with the same code and the same nodes running, the worst absolute drift over a full **ten minutes** was 0.01° — twenty samples, chassis IMU and the gimbal’s mechanical joint encoder in agreement throughout.

Take static measurements with the robot on the floor. If a table is unavoidable, re-check the drift there rather than assuming it carries over.

If it drifts on the floor too, then it is worth looking further — but these were each tested and found *not* to be the cause, so they can be skipped:

- The manual chassis command loop. It brakes once when idle and then sends nothing; confirm with the [MANUAL-DRIVE] log line, which is absent when no command is being sent.
- gimbal_lead robot mode (“chassis follows gimbal”), in which the chassis ignores the yaw component of a drive command. The node sets the mode at startup and reads it back — see the [MODE] line, which reports what the robot actually returned rather than what was requested.
- Reaction torque from the gimbal. Ten sharp 35° slews moved the chassis by 0.02° in total.
- The gimbal being off-centre. Deliberately offsetting it by 32.5° produced no chassis rotation over a minute.

Found 2026-08-17, likely the real mechanism: chassis_lead robot mode itself. Not on the list above — it was not yet the default when that investigation ran. With the node up, zero commands on `/carolus/cmd_vel` (confirmed absent from the log), and the robot’s wheels held **clear of any surface**, the wheels were observed turning continuously. No ground contact, so this cannot be a friction or surface-level effect; it is an uncommanded drive signal, full stop. Switching the robot’s mode from `chassis_lead` to `free` while running stopped it immediately. **Not yet isolated by a dedicated experiment** — the working explanation is that `chassis_lead`’s own firmware loop holds the chassis against the same drifting attitude estimate (Section 14.3) that used to drive the gimbal in `free` mode, one level down. If so, this also explains why 2026-08-12’s table-vs-floor result did not reproduce later on the floor alone (Section 14.3 already flags that contradiction): the wheels may have been turning in both cases, translating into visible platform rotation only where the surface had enough grip to carry it. **The default robot mode is free as of 2026-08-17 for exactly this reason** — `RM_ROBOT_MODE=chassis_lead` still exists as an override but is no longer recommended for any use, including the driving case `chassis_lead` was originally chosen for, since Section 14.3’s chassis-referenced hold now solves that problem in `free` mode directly.

Do not diagnose this from the chassis IMU alone. It is not reliable on this robot (Section 14.5 documents it reading -72.8° of roll on a robot standing level). Cross-check with the gimbal joint encoder (`/carolus/gimbal_yaw_rel`), which is a mechanical measurement of the chassis-to-gimbal angle: if the chassis turns while the gimbal holds its heading, the two must move by equal and opposite amounts. A camera-based check works too, but only in a lit room — in the dark the frame carries no texture and the correlation returns noise, which reads as hundreds of pixels of phantom motion.

14.5 The chassis ignores every command while everything else works

A distinct and much more confusing symptom than the one above: telemetry keeps streaming, `/odom` keeps publishing, the camera works, the *gimbal* obeys — and the wheels do nothing. No error is raised anywhere: `chassis.drive_speed()` is called, returns normally, and the robot simply does not move.

Check the robot’s own attitude and status flags first. They are printed by `rm_cam_beacon.py` (Chapter 7) on every loop:

```
1 [STATUS] pickup=0 slip=0 roll=1 slope=1 impact=0
2 [ATTI]   yaw=-141.4 pitch=-10.3 roll=-72.8
```

Measured on this robot, 2026-08-12. `roll=1` in `[STATUS]` is the chassis’ *roll-over* flag, and the `[ATTI]` line shows why it is set: the chassis attitude reference reads -72.8° of roll. An S1 that believes it has rolled over inhibits chassis motion as a safety behaviour, and it does so *without reporting anything* through the SDK — which is why the call succeeds and nothing happens.

Confirm it is the reference, not the robot. Look at the camera image: the S1’s gimbal has no roll axis, so a chassis genuinely lying at 70° produces visibly canted frames. A level horizon with `roll` reading tens of degrees means the attitude reference is wrong, not the robot.

The power-cycle clears the inhibition, but not the underlying bias — and the two are easy to confuse. Measured 2026-08-12, after a power-cycle restored normal chassis motion: [STATUS] reported roll=0 (the roll-over latch had cleared, so the wheels responded again) while [ATTI] still read pitch=-16.6 roll=9.1 on a robot standing flat — and read *exactly* those values on every sample for minutes, unchanging. A constant offset is not a drifting sensor: it points at a calibration or mounting bias, not a failing gyro. Practical consequence: **treat [ATTI] pitch and roll, and the slope flag, as unreliable on this robot** even when the chassis is working. Yaw was stable throughout and is the channel to use for heading.

Remedy. The S1 establishes its attitude reference at power-on, and the SDK exposes no calibration entry point (**verified:** the installed `robomaster` package offers only `reset`, `sub_mode` and `unsub_mode` on `chassis`, nothing that re-zeroes attitude). Power the robot down, stand it flat and still on a level surface, power it back on without moving it, and confirm [ATTI] reports a roll near zero and [STATUS] reports roll=0 before trusting any motion.

Any chassis measurement taken while this flag is set is void — not merely noisy. Odometry keeps updating and commands keep being accepted, so a logged run looks superficially valid while the wheels never turned. Check [STATUS] at the start of a session, not after a result looks strange.

14.6 The Pi will not turn on

The cable is almost certainly in the Pi's own POWER IN port instead of the UPS HAT's port underneath. Check that the HAT's blue charge LED is lit (Section 5.1).

14.7 rostopic from the lab PC finds nothing

1. **Is the node actually running?** On the Pi: `pgrep -af "[r]m_cam_beacon"`.
2. **If it is running but invisible from the PC**, the shell that launched it was missing `ROS_MASTER_URI/ROS_IP` — `rospy` falls back to `localhost` and the node is reachable only from the Pi itself. Relaunch with both exported (Chapter 7). This is the single most common cause.
3. **Is your own shell pointed at the right master?** On the PC: `echo $ROS_MASTER_URI` should read `http://192.168.0.103:11311`.
4. **A black image or an AF_INET error** instead of no topic at all is hostname resolution — add the Pi to `/etc/hosts` on the PC (Chapter 6).

14.8 The camera publishes but /pose never does

1. **Is a beacon actually in view, powered on?** Check [BEACON] `status=` in the T2 log, or the launcher's beacon indicator. [DUTY] `beacon in view 0.0%` confirms it is not.
2. **Is Carolus running?** On the Pi: `pgrep -f carolus_astrobees`.
3. **Are the hue bounds right for your LEDs?** This is the classic silent failure: the node runs, looks healthy, and rejects every blob. `lb_hue/ub_hue` are on **OpenCV's 0–180 scale**, not 0–360 — entering 240 for blue rejects everything. See the end of Chapter 9.

4. **Are all four LEDs visible?** P4P needs four points; three is not enough and produces nothing rather than a degraded pose.

14.9 The keyboard does not drive the robot

1. **Is the launcher in MANUAL mode?** The button must read `MODE: MANUAL` in orange (Chapter 11).
2. **Move the mouse over the launcher window** — an external terminal opened by T1/T3 has probably stolen keyboard focus.
3. **Numpad specifically dead?** LOCK is on and holds the gimbal exclusively. The chassis keys should still work.

14.10 /imu is silent

Do not suspect the robot first. If a ROS publisher is created *after* its SDK subscription, the first callback raises inside the SDK's dispatcher thread and kills it — after which no telemetry callback ever fires again, with the topic still advertised and nothing logged. See the second warning box in Chapter 7.

14.11 The build fails

On the Pi (Ubuntu 20.04) the build is standard; a failure there is usually memory — pass `-j2`. On a lab PC running Ubuntu 22.04, the five fixes in Section 9.6 are all required, and a segfault at launch rather than at build time is Fix 5 (the `LD_PRELOAD`).

14.12 A plain python3 script dies on import cv2

```
1 AttributeError: _ARRAY_API not found
```

numpy shadowing, lab PC only. A newer numpy (2.2.6) installed into `~/.local/lib/python3.10/site-packages/` by an unrelated pip install shadows the system numpy that apt's `python3-opencv` was actually compiled against, for any bare `python3` invocation — `python3 -c "import numpy; print(numpy.__file__)"` shows which one wins. **Fix:** set `PYTHONNOUSERSITE=1` before running any script that imports `cv2` via plain `python3` on this machine. Not needed inside a ROS launch (which does not import the user site-packages the same way) or on the Pi (no such conflicting install there).

14.13 Messages you can safely ignore

Three appear routinely on a healthy system and have each cost someone an afternoon:

- `bash: .../catkin_ws/devel/setup.bash: No such file or directory` — a leftover line in a shell profile. Harmless.
- `WARNING: disk usage in log directory [...] is over 1GB` — accumulated ROS logs. `rosclean purge` when convenient.

- `imageio 2.35.1` has requirement `pillow>=8.3.2`, but you'll have `pillow 7.0.0` — from `myqr`, a transitive dependency of the SDK used only for Wi-Fi QR codes, which this RNDIS setup never uses.